

**Manouba University
National School Of Computer Sciences**



PCD report

**Anomaly Detection in IOT networks
using machine learning techniques**

Realized by:

**Omar Ayach
Ghassen Jebali
Mahdi Siala**

Supervized by:

Chiraz Houaidia

Academic Year:
2024-2025

Supervisor's signature

Abstract

This project presents an intelligent anomaly detection system designed for IoT networks using machine learning techniques. The goal is to improve network security by identifying unusual behaviors in data traffic that may signal cyber threats. The system was developed using the Django framework for the backend and various machine learning models for classification. After preprocessing a benchmark IoT dataset, we trained and evaluated multiple algorithms, ultimately selecting a model that offered high accuracy and low false-positive rates. The results demonstrate that the proposed approach can effectively detect anomalies in real time, offering a scalable and practical solution for modern IoT environments.

KeyWords: Iot, Anomaly Detection, Autoencoders

Acknowledgments

We would like to express our sincere gratitude to all those who supported and guided us throughout the realization of this end-of-year project.

First and foremost, we extend our deepest thanks to our supervisor, **Mrs. Chiraz Houaidia**, for her invaluable guidance, patience, and continuous encouragement. Her expertise and insightful feedback were instrumental in shaping the direction and success of our work.

We also wish to thank the faculty and staff of the **National School of Computer Sciences (ENSI)** for providing us with the academic environment and resources necessary for this project.

Finally, we are grateful to our families and friends for their unwavering support, motivation, and understanding throughout this journey.

Omar Ayach, Ghassen Jebali, Mahdi Siala

List of Acronyms

IoT Internet of Things

ML Machine Learning

SVM Support Vector Machine

RNN Recurrent Neural Network

GAN Generative Adversarial Network

VAE Variational Autoencoder

DAE Denoising Autoencoder

RAE Recurrent Autoencoder

CAE Convolutional Autoencoder

LSTM Long Short-Term Memory

GRU Gated Recurrent Unit

RMSE Root Mean Square Error

AUC Area Under the Curve

ROC Receiver Operating Characteristic

ICS Industrial Control System

GPU Graphics Processing Unit

NaN Not a Number

DDoS Distributed Denial of Service

IP Internet Protocol

AI Artificial Intelligence

CSV Comma-Separated Values

HTML HyperText Markup Language

CSS Cascading Style Sheets

JS JavaScript

SQLite SQLite Database

PostgreSQL PostgreSQL Database

MSE Mean Squared Error

XGBoost eXtreme Gradient Boosting

Contents

General Introduction	1
1 General Context	2
1.1 Context and Problem	2
1.2 Limitations of traditional methods:	5
1.3 Proposed Solution	6
1.4 Project Objectives	7
1.5 Work Methodology	7
1.6 Conclusion	10
2 State of the art	11
2.1 Anomaly Detection in IoT Systems	11
2.1.1 Definition of an anomaly in an IoT context	11
2.1.2 Categories of Anomalies in IoT	12
2.1.3 Traditional Methods for Anomaly Detection	13
2.2 Machine Learning for Anomaly Detection	13
2.2.1 Supervised Methods	13
2.2.2 Unsupervised Methods	14
2.3 Autoencoders: Theory and Applications	15
2.3.1 Working Principle of Autoencoders	15
2.3.2 Variants of Autoencoders	16
2.3.3 Autoencoders in Anomaly Detection	17
2.4 State of the Art	18
2.4.1 Existing Studies on Autoencoders in IoT Systems	18
2.4.2 Identified Limitations in the Literature	19
2.4.3 Synthesis	20
2.5 Conclusion	20
3 Design of the Proposed Solution	21
3.1 Data Pre-processing	21
3.1.1 Data Description	21

3.1.2	Data Cleaning and Normalization	24
3.2	Model Training	25
3.2.1	Data Partitioning	26
3.2.2	Hyperparameter Optimization	26
3.3	System Design	28
3.3.1	Global Use Case	29
3.3.2	Overall Architecture	29
3.3.3	User Interaction Flow	32
3.4	Conclusion	35
4	Experimental Results	36
4.1	Development Environment	36
4.1.1	Hardware Environment	36
4.1.2	Software Environment	37
4.2	Autoencoder Results	38
4.3	Anomaly Detection and Classification	39
4.3.1	Anomaly Injection and Labeling	39
4.3.2	Reconstruction-Based Anomaly Detection	41
4.3.3	Anomaly Classification Techniques	41
4.3.4	Summary	48
4.4	The Software Implementation	49
4.4.1	Technologies Used	49
4.4.2	Implementation of the Dashboard	49
4.5	Conclusion	51
	General Conclusion	52
	Bibliography	54

List of Figures

1.1	Evolution of the Number of IoT Devices Worldwide (Year by Year) [1]	3
1.2	CRISP-DM Methodology	8
2.1	Taxonomy of Machine Learning Methods	15
2.2	Autoencoder architecture	16
2.3	Anomaly Distribution Graph	17
3.1	Sample View of the Sensor Database	22
3.2	Statistical Summary of collected data from sensors	23
3.3	Temperature Distribution for Sensor 1	23
3.4	Temperature Distribution for Sensor 2	24
3.5	Temperature Distribution for Sensor 3	24
3.6	Normalization of Features Using StandardScaler	25
3.7	Dataset Splitting into Training, Validation, and Testing sets	26
3.8	Training Loss of the Autoencoder Over Epochs	28
3.9	Use case diagram	29
3.10	2-tier architecture[2]	30
3.11	3-tier architecture[2]	31
3.12	Sequence Diagram: Room Status Display on Dashboard	33
3.13	Sequence Diagram: Viewing Detailed Room Information	34
3.14	Sequence Diagram: Viewing Historical Anomalies	35
4.1	Distribution of reconstruction errors.	38
4.2	Anomaly Injection and Labeling	40
4.3	Frequency of Each Classified Anomaly Type	40
4.4	Autoencoder Reconstruction and Error-Based Features	41
4.5	Real vs. Predicted Anomalies using K-Means	42
4.6	Predicted Anomalies using Random Fores	43
4.7	calculate metrics	45
4.8	Two-Stage Anomaly Classification Architecture (XGBoost + SVM)	47
4.9	XGBoost Confusion Matrix (Stage 1)	48
4.10	SVM Confusion Matrix (Stage 2)	48

4.11	Login Interface	50
4.12	Login Error Message	50
4.13	Main Dashboard	50
4.14	Room page	51
4.15	Anomaly Detection History	51

List of Tables

2.1	Comparison of Autoencoder-Based Anomaly Detection Studies in IoT Systems	19
3.1	Impact of Different Batch Sizes	27
3.2	Impact of Different Learning Rates on Model Performance	28
4.1	Classification Performance Metrics	45

General Introduction

The rapid proliferation of the Internet of Things (IoT) has led to a significant increase in the deployment of connected devices across diverse domains such as healthcare, industrial automation, smart homes, and environmental monitoring. These devices continuously generate vast volumes of heterogeneous data, whose integrity and reliability are crucial for the correct functioning of IoT systems. However, the dynamic and distributed nature of IoT environments makes them prone to various types of anomalies, ranging from sensor faults and connectivity issues to cyber-attacks. Detecting such anomalies in real-time is critical to ensuring security, availability, and performance.

Traditional rule-based and supervised approaches are often limited by their dependency on labeled data and lack of scalability in complex environments. Autoencoders, a class of neural networks capable of unsupervised representation learning, have emerged as promising tools for modeling normal behavior and identifying deviations indicative of anomalies.

This report explores the use of autoencoder architectures for detecting and classifying anomalies in IoT systems. It focuses on leveraging their reconstruction capabilities and latent space representations to identify abnormal patterns without relying on extensive labeled datasets.

The remainder of this report is organized as follows:

- **Chapter 1** provides the general context of anomaly detection in IoT systems, including the definition and types of anomalies typically encountered.
- **Chapter 2** presents a detailed state of the art on autoencoders, reviewing their key variants and applications in anomaly detection.
- **Chapter 3** describes the design and implementation of the proposed solution, including data preprocessing, the anomaly detection pipeline, simulation of anomalies, and anomaly classification strategies.
- **Chapter 4** presents the results and performance evaluation of the implemented models using various metrics and benchmarks.
- **The report concludes** with a summary of findings, limitations, and directions for future research.

Chapter 1

General Context

Introduction

The Internet of Things (IoT) connects a wide range of devices—such as sensors, actuators, and embedded systems—to collect and share data. This technology plays an important role in many areas, including smart homes, healthcare, agriculture, and industry. As the number of connected devices continues to grow, maintaining the reliability and security of these systems becomes a major concern.

Failures in IoT systems, whether due to hardware faults, network issues, or unexpected behavior, can have serious consequences. Detecting these anomalies early is essential to avoid data loss, service interruptions, or safety risks.

This project focuses on detecting anomalies in IoT systems using machine learning techniques. In particular, it explores the use of autoencoders to identify unusual patterns in data and help ensure system stability and performance.

1.1 Context and Problem

IoT stands for the "Internet of Things," which is a wide network of connected devices that includes software, sensors, hardware, electronic components, and unique identifier codes. IoT devices can interact and function autonomously, without the need for human intervention, allowing efficient data transfer and communication between all connected devices [3].

From small structures such as wearables and smart homes to larger, more complex systems such as factories, farms, and even entire cities, this technology has changed the way we live and work. According to IoT Analytics [4], the number of connected IoT devices was around 16.6 billion at the end of 2023, and this number is predicted to increase by 18% by the end of 2024 to reach 18.8 billion and exceed

41 billion by 2030. Thus, IoT networks play an important role across different fields such as medicine, manufacturing, farming, etc.

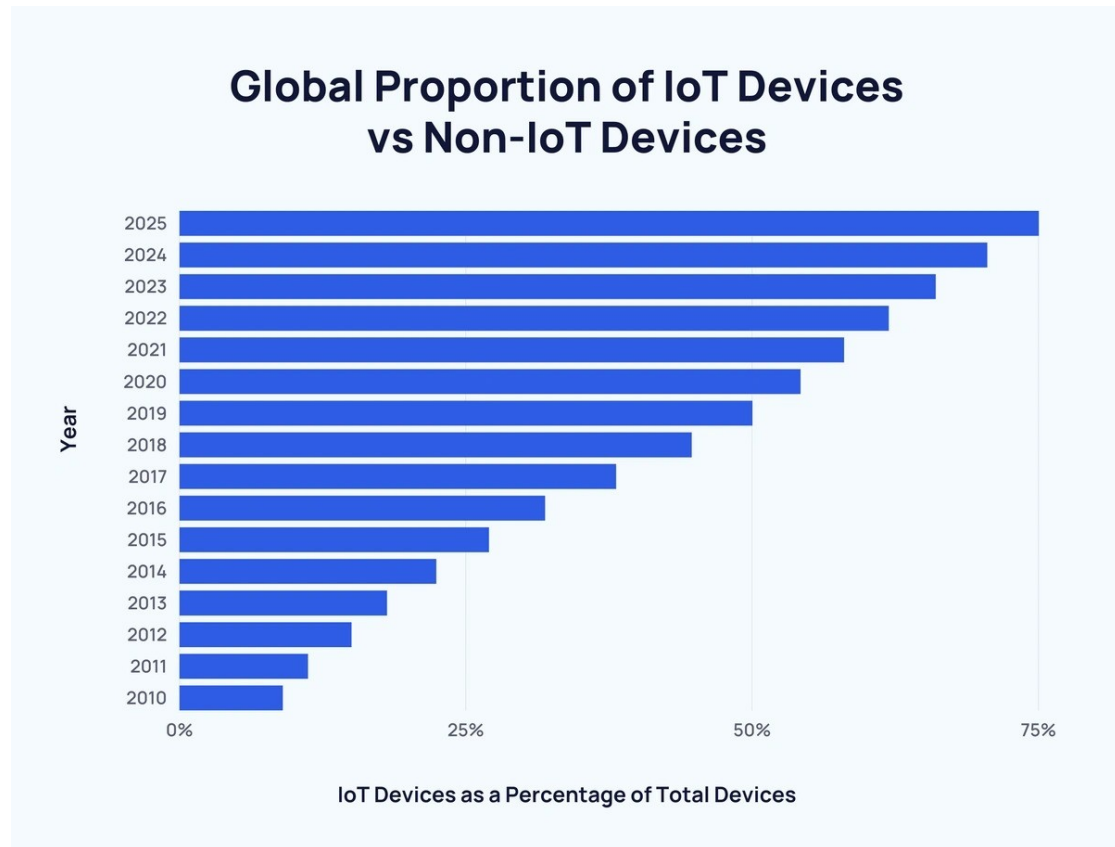


Figure 1.1: Evolution of the Number of IoT Devices Worldwide (Year by Year) [1]

Key applications of IoT include:

- **Industrial IoT (IIoT):** Equipment monitoring, predictive maintenance, smart factories.
- **Smart Homes:** Thermostats, lighting systems, smart locks, voice assistants.
- **Healthcare:** Wearable devices, remote patient monitoring.
- **Agriculture:** Soil moisture sensors, automated irrigation.
- **Transportation:** Fleet tracking, smart traffic systems. [5]

As these systems become more integrated into our world, trials and errors are bound to occur. Managing the amount of data these systems generate is a challenge in itself. They also need to be reliable, scalable, efficient, secure, and cost-effective.

And like any useful resource, IoT systems need to be sustainable and meet the expectations and goals set for them. Any downtime or failure can be costly in any application sector and catastrophic for critical sectors such as healthcare, transportation, or agriculture.

Anomalies in IoT systems and networks refer to deviations from normal behavior, often caused by hardware malfunctions, software bugs, cyberattacks, or environmental factors. These irregularities can disrupt the functionality of interconnected devices, leading to degraded performance, compromised security, or system failures. Such issues can have significant consequences, including reduced reliability, increased operational costs, and potential safety hazards in critical applications like healthcare, transportation, or smart grids. Identifying and classifying these anomalies is essential for understanding their root causes, mitigating their impact, and maintaining the overall health of IoT systems. A systematic approach to anomaly detection ensures improved system performance, optimized resource usage, and enhanced security, enabling the effective utilization of IoT technology across various domains.

However, classifying anomalies in IoT systems presents several challenges due to the diversity and heterogeneity of IoT devices, protocols, and applications. The wide variety of devices, ranging from sensors and actuators to complex embedded systems, generates diverse data types and patterns, making it difficult to establish universal classification criteria. The dynamic and distributed nature of IoT networks further complicates anomaly detection, as devices often operate under varying conditions and workloads.

Anomalies in IoT systems can be categorized into different types based on their origin, each impacting the system's functionality and reliability in unique ways. In this work, we propose to consider the following classification:

1. **Class 1: Sensor-related anomalies** : include hardware issues such as sensor malfunctions (e.g., broken sensors providing no data), software glitches causing incorrect readings, and energy-related anomalies like battery depletion leading to intermittent operation. For instance, in a smart agriculture system, a malfunctioning soil moisture sensor could provide erroneous data, resulting in over or under-irrigation.
 - **Hardware:** There is a problem with hardware or the installation of the sensor. So either the sensor itself is broken or it was deployed incorrectly.
 - **Software:** Something is wrong with the software or the deployment configuration.
 - **Energy:** There is a significant unexplained power drain.

2. **Class 2: Network-related anomalies** involve issues in communication and cybersecurity. Communication anomalies, such as packet loss, delays, or bandwidth congestion, can disrupt data transmission between IoT devices, leading to incomplete or outdated information. Cyberattack anomalies, such as denial-of-service (DoS) attacks, data spoofing, or unauthorized access, compromise the network’s integrity and security. For example, in a smart home setup, a DoS attack on a thermostat’s control system could prevent temperature adjustments.

- **Communication:** There is a connectivity problem example: not receiving communication, receiving incomplete data (packet loss) or receiving them but with a significant delay.
- **Cyber attack:** There is an unauthorized outside party acting maliciously inside the system.

3. **Class 3: Application-related anomalies** pertain to deviations in expected application behavior, such as incorrect data aggregation, decision-making errors, or system crashes. For instance, an anomaly in a predictive maintenance system could cause it to miss critical warnings about equipment failure.

Identifying and addressing these anomaly types is essential for ensuring the robust operation and security of IoT systems. These vulnerabilities can lead to economic losses, safety hazards, and data breaches. Hence, it is crucial to detect anomalies early, whether they are caused by technical faults or malicious activities.[6]

1.2 Limitations of traditional methods:

Given the diverse and dynamic nature of IoT environments, various methods have been developed to identify anomalies effectively.

These methods can be broadly classified into statistical approaches, geometric techniques, machine learning (ML)-based methods and deep learning models. Each category offers unique strengths, ranging from simplicity and interpretability to the ability to handle complex and high-dimensional data.

Traditional detection methods (such as rule-based systems or fixed thresholds) often fall short because:

- They lack adaptability to evolving data patterns.
- Require manual configuration and domain expertise.
- Cannot handle large-scale, high-dimensional IoT data.

- Produce high false positive/negative rates. [7]

Improved anomaly detection methods in IoT systems can be categorized into four broad approaches, each with distinct strengths and limitations:

1. Classic Methods (Statistical & Geometric):

- *Statistical models* (e.g., Z-score, probability distributions) identify anomalies based on deviations from expected patterns, offering simplicity but struggling with complex data.
- *Geometric techniques* (e.g., KNN, DBSCAN) leverage spatial relationships and density, suitable for high-dimensional data but computationally costly.

2. Machine Learning (ML)-Based Methods:

- Includes *One-Class SVM*, *Local Outlier Factor (LOF)*, and *Isolation Forest*, which balance interpretability and adaptability. These methods excel in handling non-linear patterns but require careful parameter tuning.

3. Deep Learning Models:

- Advanced techniques like *Graph Neural Networks (GNNs)* and *RNNs/CNNs* model complex, high-dimensional data (e.g., sequential or graph-structured). While powerful, they demand large datasets and significant computational resources.

Each category addresses different IoT anomaly detection challenges, from interpretable statistical thresholds to scalable deep learning solutions. The choice depends on data complexity, resource constraints, and application requirements.

1.3 Proposed Solution

Based on the classification we previously conducted on the methods of anomaly detection in an IoT system and with respect to the data and available computing resources, we opt for the use of Machine learning techniques as a powerful tool for the detection and classification of anomalies. In fact, Machine learning-based anomaly detection offers several key advantages over traditional methods:

- Models can learn normal patterns of behavior from data.
- They automatically detect deviations, even subtle or unknown ones.

- ML systems can adapt over time as the system evolves.
- More scalable and accurate than traditional approaches.

Among the various machine learning techniques available for anomaly detection, autoencoders offer distinct advantages for IoT systems [?][?]. These neural networks are particularly well-suited for learning compressed representations of normal behavior and identifying deviations. Key reasons for this choice include their ability to handle high-dimensional data efficiently, detect subtle anomalies through reconstruction error analysis, and operate in an unsupervised manner—critical for IoT environments where labeled anomaly data is often scarce. Additionally, autoencoders can be adapted to different IoT data types (e.g., sequential sensor data or spatial patterns) through architectural variants like LSTM or convolutional autoencoders.

1.4 Project Objectives

The main goals of this project are as follows:

- Detect and predict anomalies in an IoT system using autoencoders: Autoencoders, a type of neural network, are capable of learning compact representations of normal system behavior. Anomalies can be detected by measuring reconstruction errors, making this technique well-suited for unsupervised anomaly detection in complex IoT environments.
- Compare the autoencoder-based approach with other anomaly detection methods: This includes benchmarking performance (e.g., accuracy, precision, recall, F1-score) against traditional techniques such as statistical models, rule-based systems, and other machine learning algorithms like Isolation Forests or SVMs.

1.5 Work Methodology

A well-established and widely used framework for guiding data-driven projects is the CRISP-DM model, which stands for Cross-Industry Standard Process for Data Mining. It offers a structured, iterative, and adaptable approach for handling data mining and machine learning tasks, and is applicable across different domains and industries.

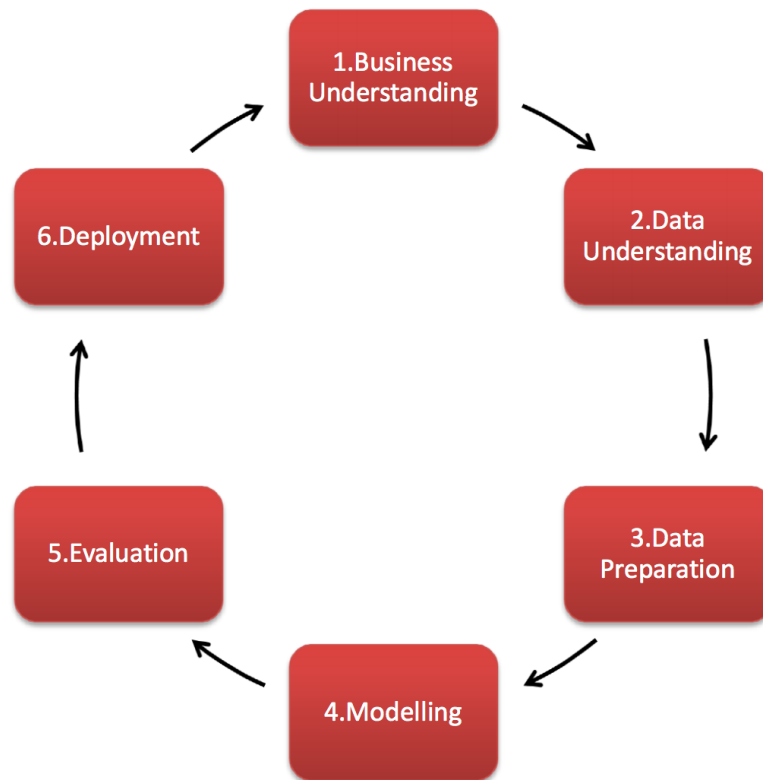


Figure 1.2: CRISP-DM Methodology

The CRISP-DM methodology is composed of six main phases:

1. **Business Understanding:** This phase focuses on understanding the project's objectives and requirements from a business or domain perspective. It involves defining the main goals, identifying potential risks, and formulating a clear data mining problem. This helps align technical goals with real-world needs.
2. **Data Understanding:** After defining the objectives, the next step is to collect initial data and begin familiarizing oneself with it. Activities in this phase include data exploration, descriptive statistics, data quality verification, and the detection of missing values or inconsistencies. It allows practitioners to assess whether the data is suitable for the problem at hand.
3. **Data Preparation:** Often the most time-consuming phase, this step involves selecting relevant attributes, cleaning the data, handling missing or noisy values, transforming variables, and creating new features. The goal is to

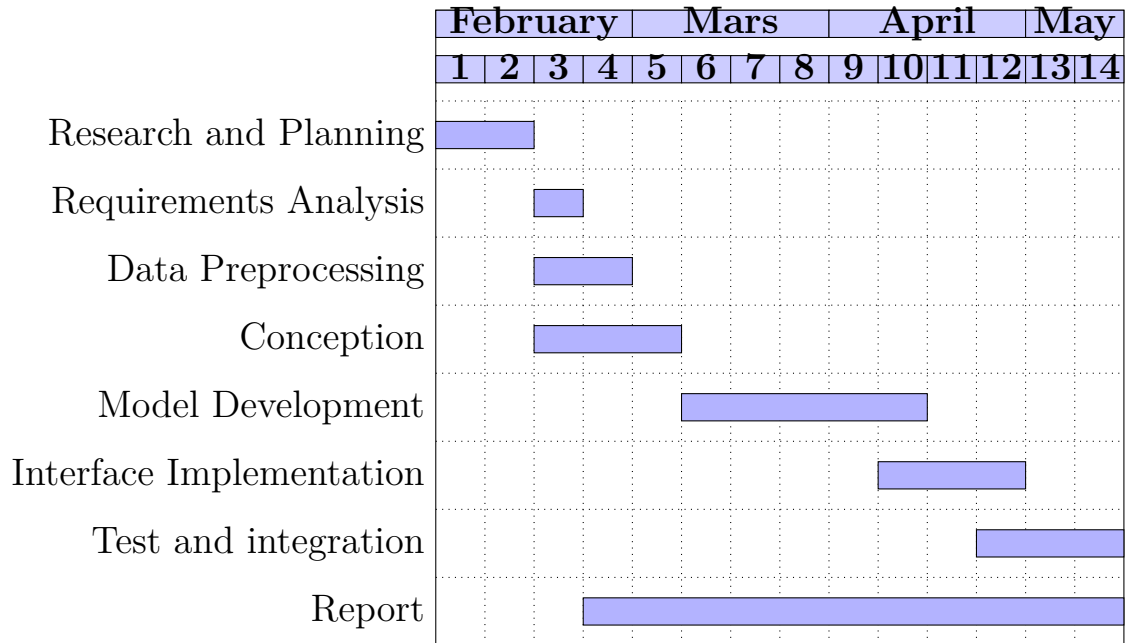
structure the data in a way that can be effectively used by machine learning or statistical models.

4. **Modeling:** In this phase, various modeling techniques are selected and applied to the prepared dataset. This may involve classification, regression, clustering, or other algorithms. Parameter tuning and model optimization are often required to achieve the best possible results.
5. **Evaluation:** Once models are built, they are thoroughly evaluated to ensure they meet the business goals defined earlier. This phase doesn't just focus on performance metrics like accuracy or precision, but also on whether the model's results make sense and are actionable from a business perspective.
6. **Deployment:** The final phase involves delivering the results in a usable format. Depending on the project, this might include deploying a predictive model into a production system, generating reports for decision-makers, or integrating insights into existing business processes.

One of the key strengths of CRISP-DM is its iterative nature: insights gained in later stages often prompt a return to earlier steps for refinement. This flexibility ensures that the project remains aligned with its objectives while adapting to new discoveries and challenges during the process.

Chronogram of the Project

The following table summarizes the timeline and major milestones of the project:



1.6 Conclusion

In this chapter, we presented the general context of IoT systems, their widespread applications, and the growing importance of ensuring their reliability and security through effective anomaly detection. We explored the challenges posed by the heterogeneity and scale of IoT data, reviewed traditional and modern anomaly detection methods, and justified the choice of machine learning, specifically autoencoders, as a suitable solution for our project. Finally, we outlined the project's objectives and the methodological framework (CRISP-DM) that will guide our work in the subsequent chapters.

Chapter 2

State of the art

The exponential growth of Internet of Things (IoT) systems has brought unprecedented connectivity, enabling smarter environments across industries such as healthcare, manufacturing, and energy. However, this increased interconnectivity also introduces complex security and operational challenges, particularly the need to reliably detect anomalies that may indicate faults, cyber-attacks, or system failures. Traditional rule-based heuristic methods, while interpretable and easy to implement, often fall short in dynamic and large-scale IoT environments due to their rigidity and limited adaptability. As a result, machine learning (ML) techniques have gained prominence, offering more flexible, data-driven approaches to anomaly detection. This chapter explores various ML strategies, distinguishing between supervised and unsupervised learning paradigms, and provides a detailed focus on autoencoders :an unsupervised deep learning model well-suited for detecting anomalies in high-dimensional IoT data. Furthermore, the chapter reviews state-of-the-art studies utilizing autoencoders in different IoT domains, highlighting both their successes and the ongoing challenges in achieving scalable, accurate, and interpretable anomaly detection systems.

2.1 Anomaly Detection in IoT Systems

2.1.1 Definition of an anomaly in an IoT context

Anomalies are patterns in data that do not conform to a well-defined notion of normal behavior [8] . Whether it's sensor values, device behavior, or network activity.

2.1.2 Categories of Anomalies in IoT

2.1.2.1 Data-Level Anomalies (Sensor Readings)

These involve the actual values reported by the sensors. Examples:

- Sudden temperature spikes (e.g., 25°C → 100°C in 1 sec)
- Negative humidity or impossible pressure readings
- Repeated identical values for a long time (stuck sensor)
- Missing or NaN values

2.1.2.2 Behavioral Anomalies (Device Actions)

The device itself behaves unexpectedly, even if the sensor values seem normal. Examples:

- Device reboots frequently
- Long periods of silence (no logs/data)
- Unusual patterns of communication
- High power consumption

2.1.2.3 Network Anomalies (Connectivity Issues)

Related to data transmission, packet delivery, and timing. Examples:

- Delayed packets (arrive too late)
- Missing data (e.g., nothing reported for 10 minutes)
- Duplicate packets (same data sent more than once)
- Unusual bandwidth usage

2.1.2.4 Security Anomalies (Cyber Threats)

Malicious or suspicious behavior of compromised devices. Examples:

- Sensor starts sending unexpected commands
- Communication with unknown IP addresses
- High-frequency data uploads (DDoS behavior)
- Tampered readings (fake or spoofed)

2.1.3 Traditional Methods for Anomaly Detection

Traditional anomaly detection techniques in IoT systems often rely on predefined rules and threshold values. These methods set fixed limits on various sensor readings, and any deviation beyond these thresholds is considered an anomaly.

- **Fixed Thresholds:** For example, in an industrial setting, a temperature sensor can have a defined normal operating range (e.g., 20 ° C to 80 ° C). If the recorded temperature exceeds these limits, the system flags it as an anomaly.
- **Statistical approaches:** Basic statistical methods such as mean, standard deviation, and percentiles are used to dynamically define thresholds. For example, a z-score (many standard deviations a data point is from the mean) can be calculated to determine if a data point is significantly different from the normal distribution.
- **Rule-Based Heuristics:** These involve manually designed rules that define abnormal behaviors. An example would be defining an anomaly if a device sends more than 'X' data packets per second, indicating potential network flooding.

While these methods are simple and interpretable, they suffer from several limitations:

- **Rigid and Static:** They do not adapt to changing environments or varying operational conditions.
- **High False Positives/Negatives:** Hard-coded thresholds may lead to frequent misclassifications of normal variations as anomalies or vice versa.
- **Scalability Issues:** Difficult to implement effectively in large-scale IoT environments with heterogeneous data sources.[9]

2.2 Machine Learning for Anomaly Detection

Given the limitations of traditional methods, machine learning (ML) has emerged as a more adaptive and efficient approach. ML models can learn patterns from historical data and detect deviations without relying on predefined rules.

2.2.1 Supervised Methods

Supervised learning requires labeled datasets where anomalies are explicitly marked. The model is trained on both normal and anomalous instances, allowing it to learn discriminative patterns. Some common supervised models include:

- Random Forest and Decision Trees: These models create branching decision structures that classify data points based on learned features
- Support Vector Machines (SVMs): Used to find a hyperplane that separates normal and anomalous instances in feature space.
- Neural Networks: Deep learning architectures can learn complex patterns in sensor data, improving anomaly detection accuracy.

Advantages:

- High accuracy when a well-labeled dataset is available.
- Ability to learn complex relationships between features.

Limitations:

- Requires extensive labeled data, which is often expensive and difficult to obtain in IoT applications.
- Struggles to generalize well when anomalies evolve or new patterns emerge.

2.2.2 Unsupervised Methods

Unsupervised methods do not require labeled data and instead detect anomalies by identifying deviations from normal behavior. These models assume that anomalies are rare events that differ significantly from the majority of observations. Common approaches include:

- Isolation Forest: A tree-based method that isolates anomalies by randomly partitioning data and identifying instances that require fewer partitions.
- One-Class SVM: Trains a model on normal data and classifies outliers as anomalies based on their distance from the learned normal boundary.
- Clustering-Based Techniques (DBSCAN, k-Means): Identify anomalies as points that do not belong to any major cluster.

Advantages:

- Does not require labeled anomalies, making it more practical for real-world IoT applications.
- Adapts to dynamic environments where anomalies evolve over time.

Limitations:

- May struggle with defining a clear boundary between normal and anomalous behavior, leading to false positives.
- Performance depends on data distribution and the chosen model parameters.

The figure below 2.1 presents a summary classification of supervised and unsupervised machine learning methods.

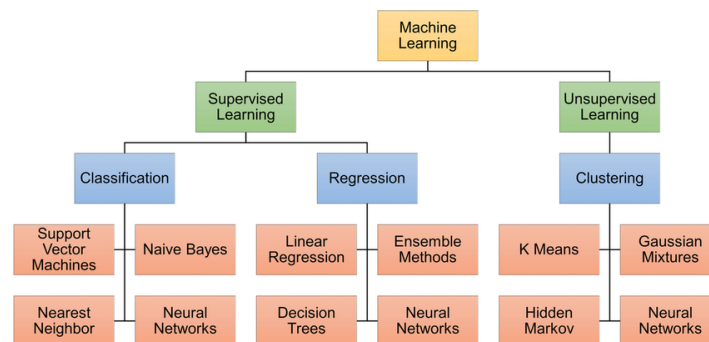


Figure 2.1: Taxonomy of Machine Learning Methods

2.3 Autoencoders: Theory and Applications

2.3.1 Working Principle of Autoencoders

Autoencoders are a type of artificial neural network designed for unsupervised learning. They consist of two main components:

- Encoder: Compresses input data into a lower-dimensional representation.
- Latent space is a lower-dimensional space that captures the essential features of the input data.
- Decoder: Reconstructs the original data from the compressed version.

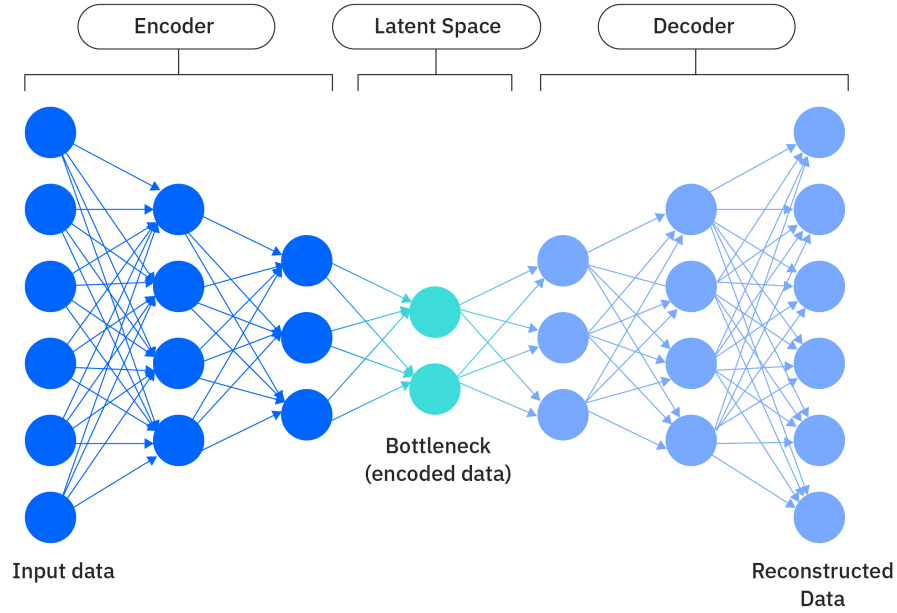


Figure 2.2: Autoencoder architecture

In anomaly detection, autoencoders are trained on normal data. When presented with an anomalous instance, the reconstruction error (i.e., difference between input and output) is significantly higher, allowing for effective anomaly identification.

2.3.2 Variants of Autoencoders

- **Denoising Autoencoders (DAE)**: Introduce noise to input data during training, making the model more robust to small perturbations and enhancing generalization.
- **Variational Autoencoders (VAE)**: Use probabilistic models to learn a latent representation, improving generalization capabilities, and enabling sampling from the learned distribution.
- **Recurrent Autoencoders (RAE)**: Incorporate recurrent layers (e.g., LSTMs or GRUs) to handle sequential data, making them effective for detecting anomalies in time-series IoT data such as network traffic and sensor logs.
- **Convolutional Autoencoders (CAE)**: Using convolutional layers instead of fully connected layers improves spatial dependence capture, making them particularly useful for anomaly detection in image-based IoT data.

2.3.3 Autoencoders in Anomaly Detection

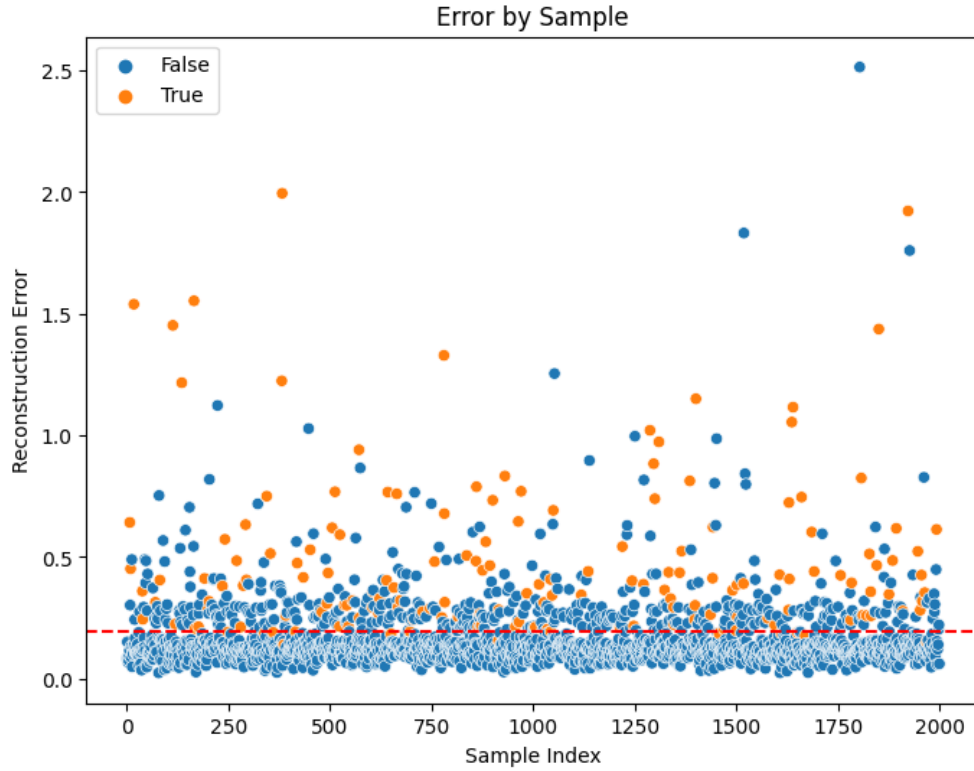


Figure 2.3: Anomaly Distribution Graph

Autoencoders are first trained exclusively on normal IoT data (e.g., sensor readings, network traffic, equipment logs). The goal is for the model to learn a compressed representation of the normal patterns. Since the autoencoder is optimized to reconstruct normal data, it becomes highly sensitive to anything that deviates from this learned distribution.[10]

2.3.3.1 Detecting Anomalies Using Reconstruction Error

After training, the autoencoder is deployed in real-time anomaly detection. Incoming data is passed through the model:

- If the reconstruction error is low, the input is considered normal.
- If the reconstruction error is high, the input is flagged as an anomaly.

2.3.3.2 Setting the Anomaly Threshold

To differentiate normal data from anomalies, a threshold is defined based on the reconstruction error:

- Statistical Methods: The threshold can be set using standard deviations .
- Percentiles: Setting the threshold at the 95th percentile of reconstruction errors in normal data.
- Adaptive Thresholding: Some approaches dynamically adjust thresholds based on recent data trends.

2.3.3.3 Advantages and Challenges

Advantages

- Can learn complex normal behavior without labeled anomalies.
- Adapts well to evolving IoT environments.
- Works well for multivariate and high-dimensional data.

Challenges

- Computationally expensive, especially for real-time edge applications.
- Sensitivity to hyperparameters (e.g., latent space size, learning rate).
- Lack of interpretability; hard to explain why a specific instance is anomalous.

2.4 State of the Art

2.4.1 Existing Studies on Autoencoders in IoT Systems

Numerous research efforts have explored the effectiveness of autoencoders for the detection of IoT anomalies. Studies have demonstrated their ability to outperform traditional rule-based methods, particularly in dynamic environments.

This section presents a review of key research works that have employed autoencoder-based models for anomaly detection in various domains. To provide a comprehensive comparison, we summarize the main characteristics of each study in a structured table, including the type and size of the dataset used, the specific variant of autoencoder implemented, the performance metrics reported, and the application context. This synthesis highlights both the versatility and

effectiveness of autoencoders in modeling complex behaviors across different IoT and industrial environments.

Table 2.1: Comparison of Autoencoder-Based Anomaly Detection Studies in IoT Systems

Ref.	Dataset (Size & Type)	Autoencoder Variant	Evaluation Metrics (Values)	Application Domain
[10]	4,100 samples, 118 features (spacecraft telemetry)	Basic Autoencoder	RMSE: 0.0025, Precision: 92.6%, Recall: 87.4%	Equipment Monitoring
[11]	60,000 time-series, 1D ECG data (length = 140)	LSTM Autoencoder	AUC-ROC: 0.94, F1-score: 0.91	Health Monitoring
[12]	SWaT dataset: 496,800 records, 51 features, 11 days	Convolutional Autoencoder	Precision: 87.1%, Recall: 84.3%, F1: 85.7%	ICS Security
[13]	XJTU-SY bearings: 23 sensors, 1Hz for 30 days	Variational Autoencoder	AUC: 0.945, Accuracy: 91.3%	Predictive Maintenance
[14]	500K+ smart grid records, 8 features, time series	Variational Autoencoder	Accuracy: 93.7%, F1-score: 0.91	Smart Energy
[15]	KDD99: 4.9M rows, 41 features	Sparse Autoencoder	AUC-ROC: 0.981, Precision: 90.2%	Network Security
[16]	Siemens dataset: 100K logs, 27 features, synthetic ICS data	Stacked Denoising Autoencoder	F1-score: 88.6%, Accuracy: 89.4%	Industrial Process Monitoring

2.4.2 Identified Limitations in the Literature

- **High Computational Cost:** Deep autoencoder architectures often consist of multiple layers and millions of parameters. Training and inference on these networks require powerful GPUs, which may not be practical for deployment on low-power edge devices.[17]
- **Hyperparameter Sensitivity:** Performance is highly dependent on the choice of architecture, learning rate, and training parameters.
- **Autoencoders, like most neural network-based models, lack interpretability.** Their decision-making process is not transparent, making it difficult

for domain experts to understand why certain data points are flagged as anomalies.

- **Generalization Across Domains:** Autoencoders trained on data from one IoT environment may perform poorly when applied to a different context with different data characteristics. This domain dependency restricts the reuse of trained models and necessitates retraining or domain adaptation.[10]
- **Latency and Real-Time Detection:** Deep architectures, especially those involving recurrent layers, introduce significant latency, which may not be acceptable for real-time anomaly detection in critical systems [18]

2.4.3 Synthesis

Anomaly detection in IoT systems is critical for ensuring the reliability, security, and efficiency of connected devices and infrastructures. While traditional rule-based methods offer simplicity, their static nature limits their effectiveness in dynamic and heterogeneous IoT environments. Machine learning, particularly unsupervised approaches like autoencoders, presents a powerful alternative by enabling models to learn normal behavior patterns without relying on labeled anomaly data. Through techniques such as reconstruction error analysis, autoencoders have demonstrated strong performance in various domains, including industrial monitoring, cybersecurity, and health diagnostics. However, significant challenges remain, including high computational costs, sensitivity to hyperparameters, lack of model interpretability, and difficulties in generalizing across different IoT environments. Future research must address these limitations to build more robust, efficient, and transparent anomaly detection systems capable of operating at scale in real-world IoT deployments.

2.5 Conclusion

In summary, this chapter has provided a comprehensive overview of machine learning strategies relevant to anomaly detection in IoT environments, with a particular emphasis on autoencoders. By examining their architecture, functioning principles, and practical implementations across various domains, we have highlighted their effectiveness in modeling complex sensor data and identifying abnormal patterns without the need for labeled anomalies. However, the review also reveals current limitations related to scalability, interpretability, and sensitivity to hyperparameters. These insights lay the groundwork for the next chapter, which introduces the proposed solution architecture and outlines how autoencoders are integrated into our anomaly detection framework.

Chapter 3

Design of the Proposed Solution

In this chapter, we will detail the design phase of our project, covering both software architecture and data preparation. The first part focuses on data design and preprocessing, crucial steps that ensure the quality and reliability of the data used for anomaly detection. The second part addresses the design of a user-friendly interface using the Django framework, chosen for its robustness and compatibility with AI projects. Through careful software planning and rigorous data preparation, we aim to build a system that is both efficient and scalable for real-world IoT environments.

3.1 Data Pre-processing

3.1.1 Data Description

For this project, we are using a combination of altered publicly available datasets and manually generated data. The basic dataset, imported from: Kaggle Dataset, was originally intended to demonstrate the use of control charts for monitoring process stability and detecting variations in real-world manufacturing scenarios.

The dataset used in this study originates from a real-world industrial roasting process. It aims to support the development of a predictive model capable of estimating product quality based on time-series data collected from a roasting machine.

The roasting machine consists of five identical chambers, each equipped with three temperature sensors. These sensors record temperature measurements at one-minute intervals, resulting in 15 temperature readings per minute across the entire machine. Additionally, two static input features—raw material layer height and moisture content—are recorded at the moment the raw materials enter the roasting process. These attributes remain constant throughout the one-hour roasting cycle

of each batch.

Product quality labels are obtained through laboratory analyses conducted on samples collected every hour at the machine's output. The corresponding ground-truth quality labels are stored in the file "data_Y.csv", which includes timestamped information matching the sample extraction times.

This dataset provides a rich basis for modeling normal operating behavior and detecting anomalies that may affect final product quality. It also supports the training and evaluation of both regression and classification models in the context of industrial process monitoring.

As shown in the figure below 3.1, this basic dataset includes :

- **Real-Time Sensor Data (sampled every minute):**
 - Temperature Sensors: 15 values (3 sensors $\times$ 5 chambers)
 - Layer Height: Time-varying thickness of raw material inside the machine
 - Humidity: Constant moisture level (measured once per batch)
- **Lab Quality Data (hourly measurements):**
 - Timestamp after each 1-hour roasting cycle
 - Quality Score in data-Y.csv

	date_time	T_data_1_1	T_data_1_2	T_data_1_3	T_data_2_1	T_data_2_2	T_data_2_3	...	T_data_4_2	T_data_4_3	T_data_5_1	T_data_5_2	T_data_5_3	H_data	AH_data	date_hour
0	2015-01-03 23:00:00	265	337	263	335	326	340	...	343	336	234	239	237	157.37	4.73	03-01-2015-23
1	2015-01-03 23:01:00	265	337	263	335	326	341	...	343	336	234	238	237	152.83	4.73	03-01-2015-23
2	2015-01-03 23:02:00	265	337	263	335	326	341	...	344	336	234	238	238	156.82	4.73	03-01-2015-23
3	2015-01-03 23:03:00	266	338	263	335	326	341	...	345	336	234	238	238	154.81	4.73	03-01-2015-23
4	2015-01-03 23:04:00	266	338	263	335	326	341	...	346	336	234	238	238	154.92	4.73	03-01-2015-23
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1751035	2018-05-03 22:55:00	253	257	256	350	350	356	...	345	341	256	267	246	153.40	8.90	03-05-2018-22
1751036	2018-05-03 22:56:00	253	257	256	350	350	356	...	345	340	256	267	246	158.97	8.90	03-05-2018-22
1751037	2018-05-03 22:57:00	253	257	256	350	350	356	...	345	339	256	267	246	156.20	8.90	03-05-2018-22

Figure 3.1: Sample View of the Sensor Database

As an initial step, we conducted an exploratory analysis of the dataset to examine the distribution of temperature values recorded by the sensors across all five chambers of the roasting machine. This analysis aimed to identify typical operating ranges as well as potential outliers or abnormal readings that may indicate faulty sensor behavior or process deviations. By visualizing the temperature distributions

using histograms and boxplots, we were able to highlight both the central tendency of the data and the presence of extreme values. This preliminary investigation was essential to understand the nature of the data and to guide subsequent steps in anomaly detection and model design. An overview of the min, max and mean values are given in figure 3.2.

	mean	std	min	25%	50%	75%	max
T_data_1_1	250.182182	32.116609	-198.000000	229.000000	250.000000	272.000000	724.000000
T_data_1_2	250.091514	30.824510	-122.000000	229.000000	250.000000	272.000000	762.000000
T_data_1_3	250.253580	30.691253	-107.000000	229.000000	250.000000	272.000000	665.000000
T_data_2_1	349.775603	42.259606	-703.000000	328.000000	350.000000	372.000000	1302.000000
T_data_2_2	349.721759	40.678867	-958.000000	328.000000	350.000000	372.000000	1179.000000
T_data_2_3	349.828165	37.984437	-191.000000	328.000000	350.000000	372.000000	889.000000
T_data_3_1	501.161070	63.257324	-775.000000	464.000000	502.000000	538.000000	1587.000000
T_data_3_2	501.109222	63.371641	-759.000000	464.000000	502.000000	538.000000	2505.000000
T_data_3_3	501.233550	62.232944	-613.000000	464.000000	502.000000	539.000000	1319.000000
T_data_4_1	349.319616	39.305621	-514.000000	327.000000	349.000000	372.000000	1177.000000
T_data_4_2	349.438980	39.199891	-471.000000	328.000000	350.000000	372.000000	1244.000000
T_data_4_3	350.029920	40.183596	-609.000000	328.000000	350.000000	372.000000	944.000000
T_data_5_1	249.748770	30.732984	-89.000000	229.000000	249.000000	270.000000	905.000000
T_data_5_2	249.631130	30.500211	-125.000000	229.000000	250.000000	271.000000	738.000000
T_data_5_3	249.693708	30.746292	-163.000000	229.000000	250.000000	271.000000	624.000000
H_data	174.716991	14.436130	141.490000	162.440000	174.400000	187.010000	207.830000
AH_data	7.499369	1.148026	2.890000	6.730000	7.510000	8.270000	11.840000

Figure 3.2: Statistical Summary of collected data from sensors

The distribution of temperature readings was analyzed separately for different sensors as illustrated by figures 3.3, 3.4 and 3.5.

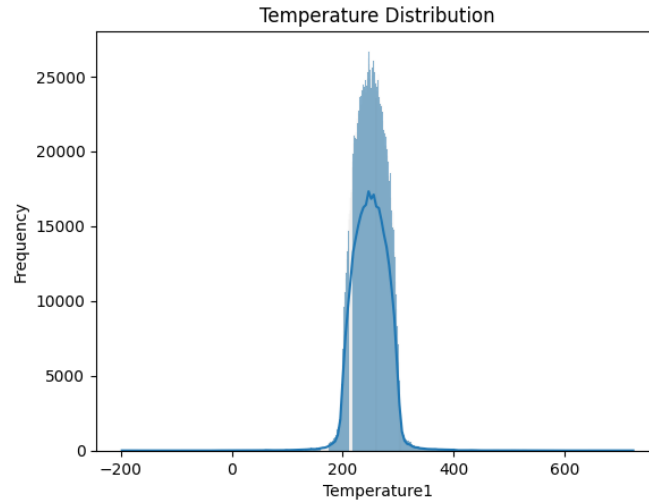


Figure 3.3: Temperature Distribution for Sensor 1

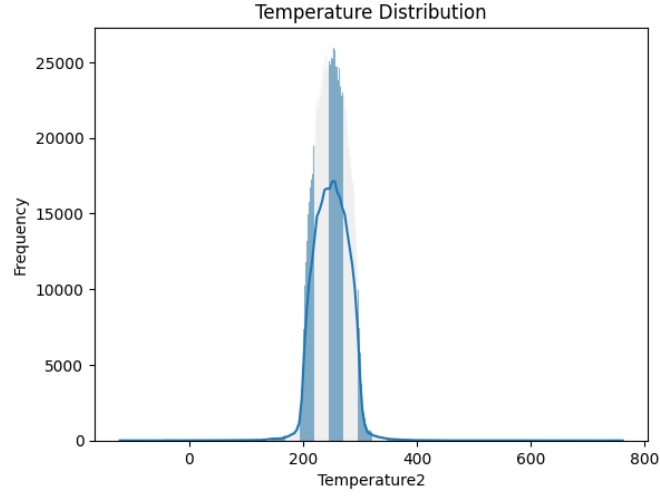


Figure 3.4: Temperature Distribution for Sensor 2

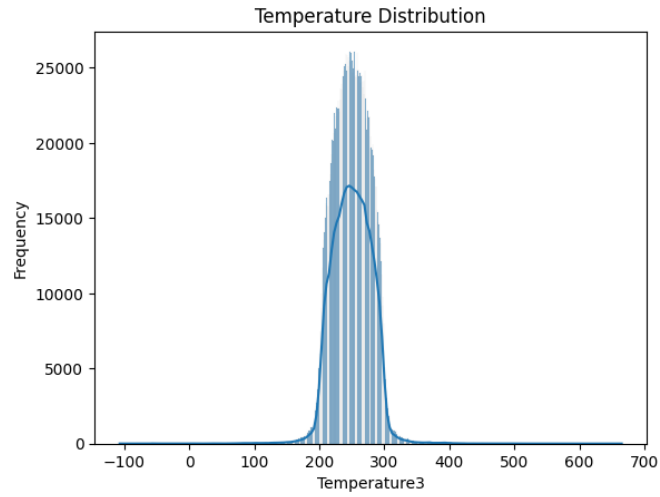


Figure 3.5: Temperature Distribution for Sensor 3

From the above histograms, it can be observed that most temperature values are concentrated around a typical operational range, with a few extreme outliers suggesting possible anomalies.

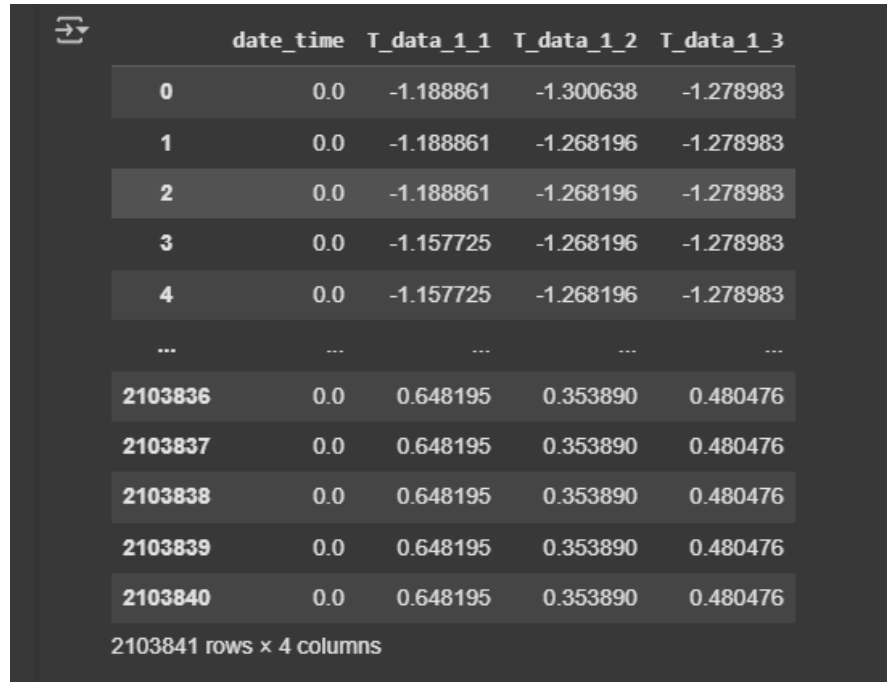
3.1.2 Data Cleaning and Normalization

To ensure the reliability and quality of the analysis, we now proceed to the pre-processing and cleaning of the dataset which consists of handling missing

values, detecting and correcting outliers, normalizing temperature measurements, and aligning timestamps between sensor data and quality labels.

The pre-processing phase involved several essential tasks aimed at refining the dataset and preparing it for anomaly detection and classification:

- **Filtering Columns:** Only relevant columns, specifically the date-time field and temperature sensor readings, were retained for analysis to reduce dimensionality and focus on meaningful variables.
- **Handling Missing Values:** Rows containing missing or incomplete data were removed to ensure consistency and avoid bias during model training.
- **Normalization:** All features were standardized using the `StandardScaler`, transforming the data to have a mean of zero and a standard deviation of one. This step was critical for aligning sensor value scales and enhancing the learning performance of autoencoder-based models as depicted in figure 3.6.



	date_time	T_data_1_1	T_data_1_2	T_data_1_3
0	0.0	-1.188861	-1.300638	-1.278983
1	0.0	-1.188861	-1.268196	-1.278983
2	0.0	-1.188861	-1.268196	-1.278983
3	0.0	-1.157725	-1.268196	-1.278983
4	0.0	-1.157725	-1.268196	-1.278983
...	...	...	...	...
2103836	0.0	0.648195	0.353890	0.480476
2103837	0.0	0.648195	0.353890	0.480476
2103838	0.0	0.648195	0.353890	0.480476
2103839	0.0	0.648195	0.353890	0.480476
2103840	0.0	0.648195	0.353890	0.480476

2103841 rows x 4 columns

Figure 3.6: Normalization of Features Using `StandardScaler`

3.2 Model Training

To train and evaluate our anomaly detection model effectively, we followed a structured approach involving data partitioning and hyperparameter tuning.

3.2.1 Data Partitioning

First, the preprocessed dataset was split, as follows, into training, validation and testing sets to allow for unbiased model evaluation. The training set was used to fit the autoencoder model, while the testing set served to assess its generalization capability.

- **Training Set:** 80%. This provides the model with sufficient examples to learn meaningful patterns, especially in unsupervised settings.
- **Validation Set:** 10%. It allows us to tune hyperparameters and prevent overfitting without biasing the test results.
- **Test Set:** 10%. It gives a final unbiased estimate of model performance.

```
[ ] from sklearn.model_selection import train_test_split

X_train, X_test = train_test_split(df, test_size=0.2, random_state=42)
X_test, X_val = train_test_split(X_test, test_size=0.5, random_state=42)
```

Figure 3.7: Dataset Splitting into Training, Validation, and Testing sets

3.2.2 Hyperparameter Optimization

To optimize the performance of our autoencoder model for anomaly detection, we performed a systematic evaluation of two key hyperparameters: batch size and learning rate. This analysis aimed to assess their impact on the model's convergence speed, training stability, and generalization capabilities, particularly in detecting rare and subtle anomalies. Through careful experimentation and observation, we identified the optimal settings that ensured both robust training and high sensitivity to anomalies.

3.2.2.1 Impact of the batch size

The choice of batch size depends on the dataset size, model architecture, and available computational resources. Based on some experiments, we selected a batch size of 50, which offered a good trade-off between training stability and model responsiveness. Smaller batch sizes, while often yielding better generalization, led to slower convergence and noisier gradients. Larger batches, on the other hand, tended to converge faster but showed signs of overfitting and reduced sensitivity to subtle anomalies. A batch size of 50 provided consistent learning dynamics, stable

reconstruction loss across epochs, and enhanced detection of both local and global deviations in the sensor data.

For our autoencoder-based anomaly detection on IoT temperature data and since we’re dealing with minute-level readings across multiple sensors over many hours or days, a batch size of 50 is a reasonable starting point.

Table 3.1: Impact of Different Batch Sizes

Batch Size	Advantages	Drawbacks
16	High gradient noise encourages exploration; better generalization	Slower convergence; less stable loss curves
32	Good trade-off between speed and generalization; smoother learning	Moderately noisy updates; slightly longer training time
50 (baseline)	Balanced gradient estimation; effective anomaly detection; reasonable training time	No significant drawbacks
128	Fast convergence; stable loss curves	Risk of overfitting; may overlook rare anomalies

3.2.2.2 Impact of Learning Rate

In terms of learning rate, we adopted a value of 0.001, which is widely recognized as a robust default when using adaptive optimizers. During initial experimentation, higher learning rates (e.g., 0.01) caused instability in the training process, leading to oscillations or divergence in the reconstruction loss. Conversely, lower learning rates (e.g., 0.0001) resulted in excessively slow convergence and minimal improvement after many epochs. The chosen rate of 0.001 provided a stable learning trajectory, enabling the model to minimize reconstruction error efficiently while preserving its ability to generalize and detect nuanced deviations in the input data.

Table 3.2: Impact of Different Learning Rates on Model Performance

Learning Rate	Advantages	Drawbacks
0.0001	Very stable updates; low risk of overshooting	Extremely slow convergence
0.001 (baseline)	Balanced speed and stability; smooth convergence	No major drawbacks observed
0.01	Rapid initial learning; fast convergence	Unstable training; may diverge; high reconstruction error

```

autoencoder = keras.Sequential([
    layers.Dense(3, activation="relu", input_shape=(4,)),
    layers.Dense(2, activation="relu"),
    layers.Dense(3, activation="relu"),
    layers.Dense(4, activation="linear")
])

autoencoder.compile(optimizer="adam", loss="mse")
autoencoder.fit(X_train, X_train, epochs=30, batch_size=128, validation_data=(X_val, X_val))
autoencoder.summary()

```

Figure 3.8: Training Loss of the Autoencoder Over Epochs

These parameters were selected after careful observation of the validation loss and the model’s sensitivity to detecting rare anomalies.

3.3 System Design

In order to enhance the usability and accessibility of our anomaly detection framework, we propose the design of a web-based interface that enables real-time interaction between end-users and the underlying anomaly prediction system. The goal of this platform is to provide an intelligent and user-friendly solution for detecting and classifying anomalies in IoT sensor networks. Through this interface, users can submit sensor data, automatically trigger the anomaly detection process using trained machine learning models, classify the nature of detected anomalies, and visualize the results in an intuitive and informative format. The user of such platform could be a system Operator whose role is to monitor sensor data in real time, and to receive anomaly alerts. In the following section, we outline the system design approach, including the definition of functional requirements, the use case diagram, the main user interaction flow, and the proposed system architecture.

3.3.1 Global Use Case

This section summarizes the main functionalities of the anomaly detection system from a user's perspective. The use case diagram 3.9 provides a high-level visual representation of the interactions between users and the system.

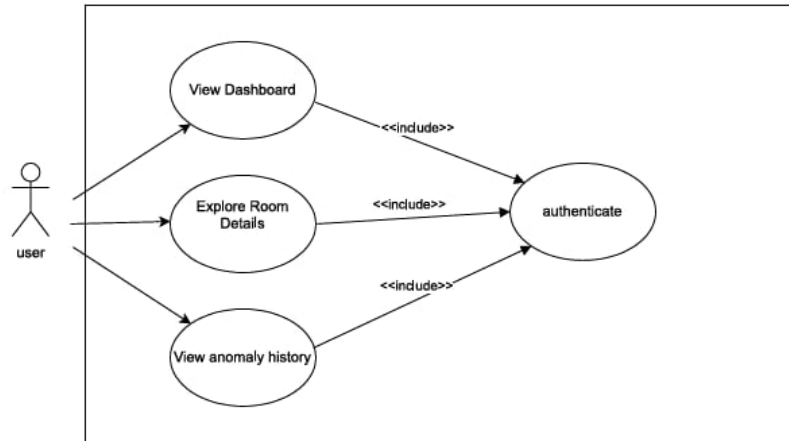


Figure 3.9: Use case diagram

The main interactions within the system include the ability to upload sensor data in a CSV file, users can initiate the anomaly detection process. The system then analyzes the data using pre-trained machine learning models to detect and classify anomalies. Users can view a dashboard summarizing the real-time status of various rooms or zones, each represented with visual indicators. By selecting a specific room, users can explore detailed graphs of sensor readings and review the history of detected anomalies. Access to these features is secured through a login system, allowing only authenticated users to interact with the platform.

3.3.2 Overall Architecture

The physical architecture directly depends on the execution context, hence the importance of making the right choice to implement our system effectively while meeting various requirements without introducing unnecessary complexity. There are several architectural approaches available; here we limit ourselves to studying 2-tier and 3-tier architectures.

3.3.2.1 2-Tier Architecture:

The 2-tier architecture aims to distribute a computerized system across remote machines using standard hardware and protocols. It characterizes client/server systems where the client requests a resource and the server provides it directly. As the name suggests, it consists of two components:

- A *client* that sends requests from a workstation.
- A *server* that stores various data related to the system's operation.

As presented in the figure 3.10, the client can handle part of the data processing, while the server manages data storage and the remaining processing—this is known as a fat client. Alternatively, the server can handle all processing, and the client merely accesses and retrieves data—this is a thin client. A rich client may also manage some local data storage.

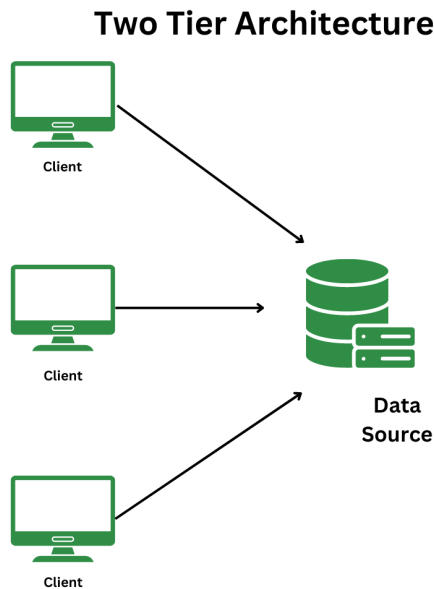


Figure 3.10: 2-tier architecture[2]

Advantages:

- It ensures loose coupling since the server is not dependent on the clients.
- Server administration and maintenance are simplified.

- This architecture provides flexibility in adding or removing clients and ensures data integrity due to its centralized structure.

Disadvantages:

- System performance mainly depends on the server.
- The cost of setting up the server is relatively high due to the required performance level.

3.3.2.2 3-Tier Architecture

The 3-tier architecture introduces an intermediate server in addition to the components of the 2-tier architecture. This server handles all the system's processing. The data is stored in a separate data server. The general structure, as visualized in figure 3.11 includes:

- A client that uses services by sending requests.
- A processing server that performs operations on the data.
- A data server where all collected detection system data is stored.

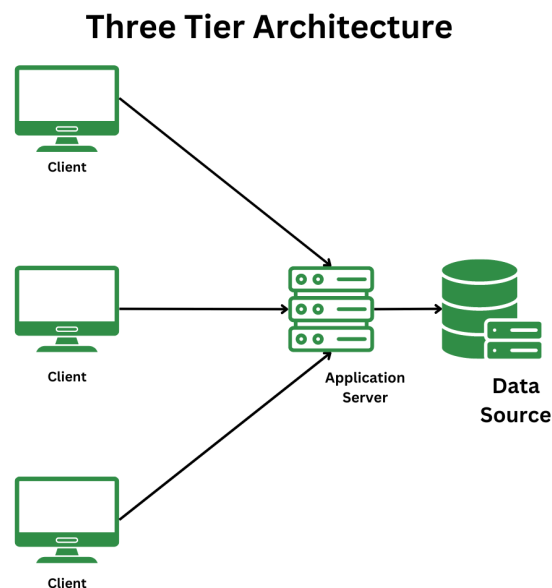


Figure 3.11: 3-tier architecture[2]

Advantages:

- This architecture simplifies deployment.
- It is more secure, as the database schema is not directly exposed due to the processing layer.
- Data handling is independent of the physical storage medium.

Disadvantages:

- Adding more layers increases the risk of errors.
- The overall cost of system setup is relatively higher.

3.3.2.3 Summary and Choice

The two-tier architecture is a client/server model where the server is versatile and directly provides all the resources requested by the client. In contrast, the three-tier architecture decentralizes server-side applications, assigning each server a specific role (e.g., web server, database server). As a result, the 3-tier architecture offers:

- Greater flexibility and scalability,
- Improved security (permissions can be assigned per service),
- Better performance (tasks are distributed).

3.3.3 User Interaction Flow

The interaction between the user and the anomaly detection platform follows a guided and intuitive workflow designed to simplify access to the system’s predictive capabilities. The typical user journey consists of the following steps:

1. **Authentication and Access Control:** Upon accessing the web interface, the user is prompted to log in using secure credentials. Based on their role (e.g., operator, analyst, administrator), users are granted appropriate access to functionalities such as data upload, model management, or system configuration.
2. **Data Input and Submission:** Users can upload historical sensor datasets or configure real-time data streams from IoT devices. The platform supports multiple file formats (e.g., CSV, JSON) and provides options to preview and validate the input data before processing.

3. **Anomaly Detection Execution:** Once the data is submitted, the system automatically initiates the anomaly detection pipeline. As described in the diagram 3.12, the user is redirected to the dashboard showing the state of each room where each room has a color-coded status (e.g., normal, warning, anomaly) indicating the presence or not of an anomaly.

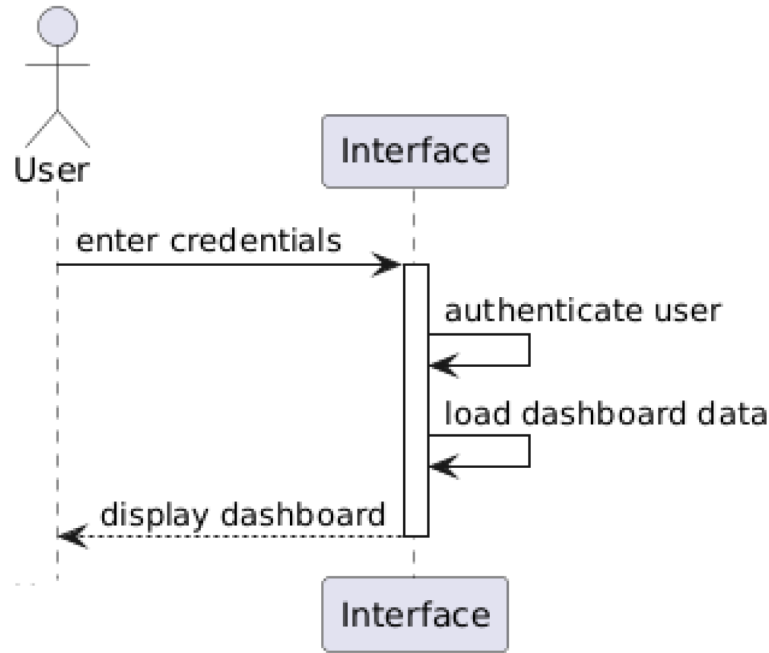


Figure 3.12: Sequence Diagram: Room Status Display on Dashboard

4. **Anomaly Classification and Analysis:** By clicking a room on the dashboard, a detailed interface will appear to the user containing:
- Graphs of sensor data over time
 - Marked anomalies
 - Historical anomaly logs

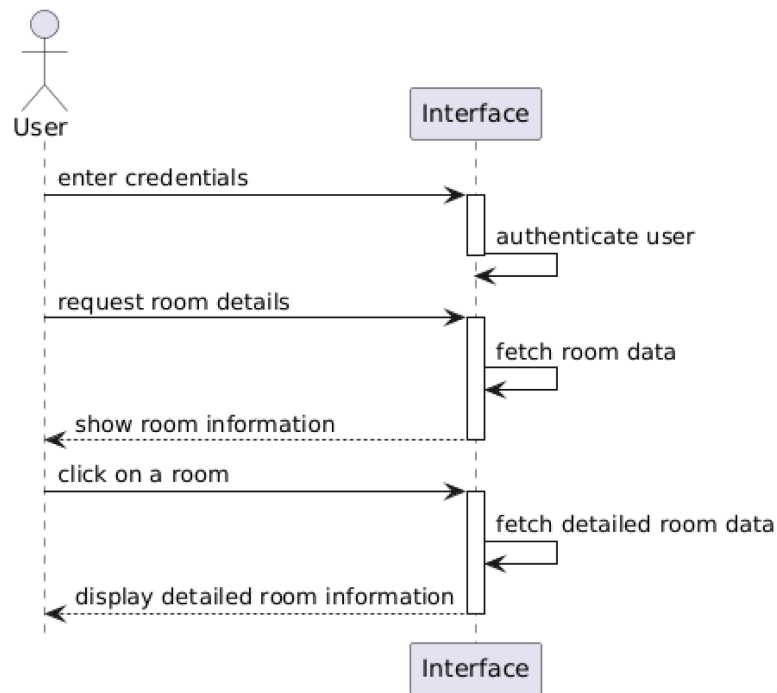


Figure 3.13: Sequence Diagram: Viewing Detailed Room Information

5. **Visualization of Results:** The user, as illustrated by the diagram 3.14 can explore historical anomaly logs and detection outcomes using a set of interactive visualizations, including time-series plots, heatmaps, anomaly timelines, and statistical summaries. User

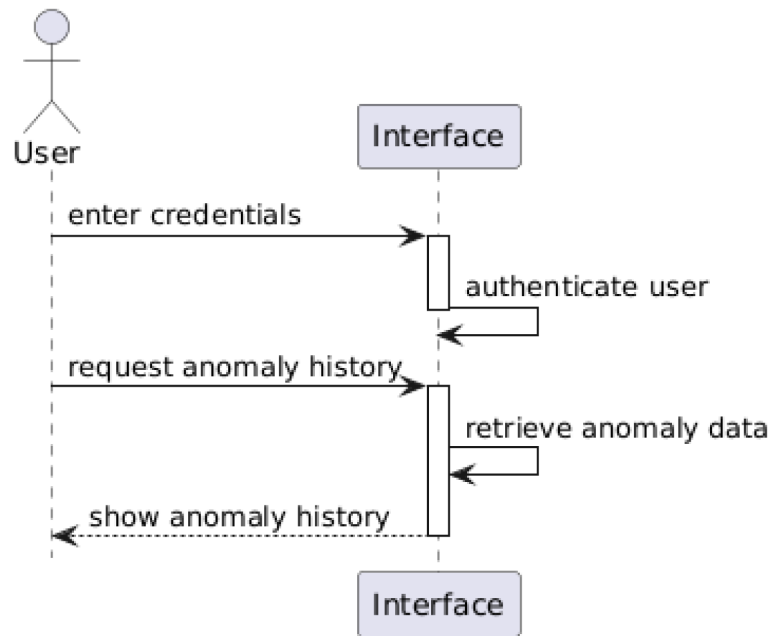


Figure 3.14: Sequence Diagram: Viewing Historical Anomalies

3.4 Conclusion

In this chapter, we first detailed the design of the data processing and modeling pipeline used for anomaly detection. The data was sourced from a real-world manufacturing context and underwent comprehensive preprocessing—such as cleaning, normalization, and visualization—to ensure its suitability for modeling.

We then explored the training phase, including how the dataset was split and the tuning of hyperparameters such as batch size and learning rate.

Second, we gave the software design details behind the web-based application that integrates the anomaly detection system and this using architectural choices, use case scenarios and user interaction flows.

Chapter 4

Experimental Results

In this section, we focus on the implementation of our system and the performance evaluation of our models. We recall that the main objective of this project is the detection of anomalies using auto-encoders and their classification using unsupervised learning models. We also developed a dashboard-type interface that allows interaction with the user and the generation of alerts in the event of a confirmed anomaly. In order to detail the steps to achieve this objective, we begin with the description of the software and hardware environment and all the technologies used; then we focus on the realization of the model with a detailed presentation of the obtained results according to some performance metrics. Then, we'll look at the different interfaces obtained and we end with a timeline displaying the progress of the project.

4.1 Development Environment

This section provides an overview of the hardware and the tools, libraries, and technologies used to implement and evaluate the proposed anomaly detection system. It includes details about the programming language, development framework, machine learning libraries, and execution environment. The choices made for the development environment aim to ensure scalability, reproducibility, and compatibility with IoT data processing workflows.

4.1.1 Hardware Environment

The project was carried out using two different laptops with the following specifications:

First Laptop

- **Processor:** Intel Core i7-11370H (11th Gen) @ 3.30 GHz
- **RAM:** 16 GB

Second Laptop

- **Processor:** AMD Ryzen 5 5600H (up to 4.2 GHz)
- **RAM:** 16 GB
- **Storage:** 1.5 TB SSD
- **GPU:** Nvidia RTX 3060 (6 GB dedicated VRAM)

4.1.2 Software Environment

The implementation of our model and data analysis was done in **Google Colab**: it is a cloud-based Jupyter notebook that runs Python code on Google's servers with no setup and access to GPUs.

The implementation of this project was done using **Python**, which is widely adopted for machine learning and data science. We used several libraries, including:

- **TensorFlow**: for building and training the autoencoder models used in anomaly detection.
- **NumPy** and **Pandas**: for data manipulation and preprocessing.
- **Matplotlib** and **Seaborn**: for data visualization and analysis.
- **scikit-learn (sklearn)**: for preprocessing tasks such as data scaling and splitting, as well as evaluation metrics.
- **Django**: a high-level Python web framework used to build the interface of the anomaly detection system.
- **Django REST Framework (DRF)**: to expose parts of our application as RESTful APIs.
- **joblib**: for saving and loading machine learning models efficiently.

These tools were chosen for their flexibility, performance, and strong community support in the field of artificial intelligence and IoT applications.

4.2 Autoencoder Results

An autoencoder is a type of neural network designed to learn an efficient representation of input data by compressing and then reconstructing it. In our approach, the autoencoder is trained exclusively on normal sensor data, representing the typical behavior of the system. The network is optimized to minimize the reconstruction error, commonly measured using the Mean Squared Error (MSE)—between the original input and its reconstruction.

During the inference phase, new sensor readings are passed through the trained autoencoder. If the input data reflects normal behavior, the model is able to reconstruct it with low error. However, if the input is anomalous—meaning it significantly deviates from the patterns learned during training—the reconstruction quality degrades, resulting in a high reconstruction error.

To detect anomalies, a threshold is defined based on the distribution of reconstruction errors observed on normal validation data. Any new input yielding a reconstruction error above this threshold is flagged as an anomaly. This technique allows the model to generalize the concept of normality and effectively identify outliers without requiring labeled anomaly data.

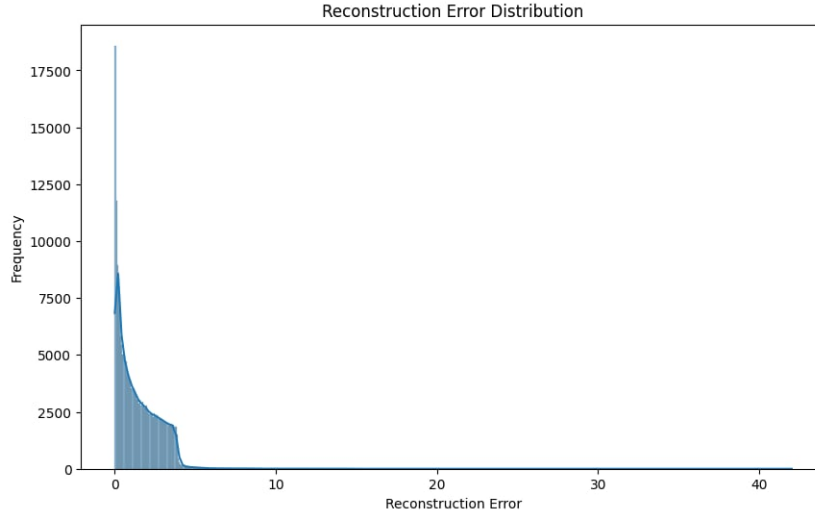


Figure 4.1: Distribution of reconstruction errors.

Figure 4.1 illustrates the distribution of reconstruction errors computed on the dataset. The x-axis represents the reconstruction error values, while the y-axis shows their frequency. As seen in the figure, the majority of samples have very low reconstruction error, with frequency sharply declining as the error increases. This right-skewed distribution confirms that the autoencoder reconstructs normal

samples well, while a small fraction of inputs produce significantly higher errors, indicating potential anomalies.

This distribution enables us to empirically select a threshold that separates normal behavior from outliers, thus making reconstruction error an effective signal for unsupervised anomaly detection.

4.3 Anomaly Detection and Classification

To evaluate the robustness of our anomaly detection system, we simulated real-world anomalies in IoT sensor data. The process was performed in three main stages:

1. anomaly injection and labeling
2. anomaly detection via autoencoder reconstruction error
3. preliminary classification analysis

4.3.1 Anomaly Injection and Labeling

We artificially introduced controlled perturbations into the test dataset to simulate realistic failure scenarios typically encountered in IoT environments. Specifically, three distinct types of anomalies were manually injected to represent different operational threats:

- **Type 0 – Network-Level Anomalies:** These anomalies simulate issues in data transmission or synchronization by programmatically modifying the timestamp field. This reflects conditions such as clock drift, delayed packet arrival, or out-of-order delivery, which are common in distributed IoT networks.
- **Type 1 – Security-Related Anomalies:** In this scenario, all sensor readings are overwritten with zero values, emulating a malicious data injection, denial-of-sensing attack, or signal suppression by a compromised node. This reflects a breach in data integrity due to cyber threats.
- **Type 2 – Sensor Faults:** Sensor-level anomalies are simulated by replacing selected temperature readings with extreme outlier values that fall well outside the learned distribution of normal data. This scenario is intended to replicate hardware malfunctions, calibration errors, or environmental noise affecting the sensors.

This anomaly injection process allows for controlled benchmarking of the model's ability to not only detect deviations from normal patterns, but also distinguish between different anomaly sources.

Each instance in the test set was labeled accordingly using an `anomaly_type` array. The full code used to generate and inject these anomalies is presented in the accompanying screenshot 4.2.

```
[ ] temperature1=list(X_test['T_data_1_1'])
    temperature2=list(X_test['T_data_1_2'])
    temperature3=list(X_test['T_data_1_3'])
    date_time=list(X_test['date_time'])
    anomaly_type=list(np.zeros(len(X_test)))
    #network failure
    for i in range(0, len(X_test)//3):
        date_time[i]=X_test['date_time'].max() + i*0.005
    #security attack
    for i in range(len(X_test)//3, 2*len(X_test)//3):
        temperature1[i]=0
        temperature2[i]=0
        temperature3[i]=0
        anomaly_type[i]=1
    #sensor failure
    for i in range(2*len(X_test)//3, len(X_test)):
        temperature1[i]=(X_test['T_data_1_1'].max() if i % 2 == 0 else X_test['T_data_1_1'].min())
        temperature2[i]=(X_test['T_data_1_2'].max() + 100)
        temperature3[i]=(X_test['T_data_1_3'].min() - 100)
        anomaly_type[i]=2
```

Figure 4.2: Anomaly Injection and Labeling

This step results in the distribution below 4.3 of anomalies by type.

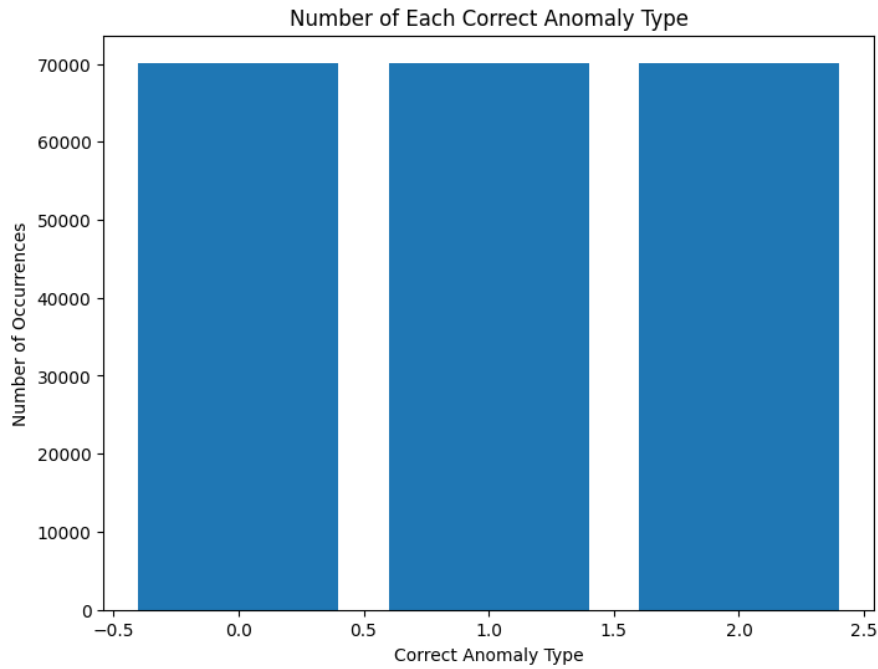


Figure 4.3: Frequency of Each Classified Anomaly Type

4.3.2 Reconstruction-Based Anomaly Detection

The modified test dataset, containing both normal and anomalous samples, was passed through the trained autoencoder. The model attempted to reconstruct the input data, and the mean squared error (MSE) between the original and reconstructed vectors was calculated for each instance.

Any sample with a reconstruction error above this threshold was labeled as an anomaly. This allowed for a first binary classification of normal vs anomalous data.

```
import numpy as np

# Ensure correct shape
input_arr = np.array(input_row).reshape(1, -1)

# Step 1: Autoencoder reconstruction
reconstructed = autoencoder.predict(input_arr, verbose=0)
error = np.mean(np.square(input_arr - reconstructed), axis=1)[0]

# Step 2: Normal or Anomaly?
if error < threshold:
    return "Normal"

# Step 3: Compute error-based features
error_features = np.array([
    np.mean((input_arr - reconstructed)**2),
    np.std((input_arr - reconstructed)**2),
    np.max((input_arr - reconstructed)**2),
    np.min((input_arr - reconstructed)**2)
]).reshape(1, -1)
```

Figure 4.4: Autoencoder Reconstruction and Error-Based Features

4.3.3 Anomaly Classification Techniques

Following the detection of anomalies using autoencoders, we applied three different classification techniques in order to choose the best technique for anomaly classification. Our objective was to distinguish between three categories of faults in IoT sensor networks, mainly (i) network failure, (ii) security anomaly, and (iii) sensor failure:

The three classifiers evaluated were K-Means (unsupervised), Random Forest, and XGBoost (both supervised). Below, we detail the approach and performance of each model.

4.3.3.1 K-Means Clustering

Motivation: K-Means clustering was selected as a baseline unsupervised method. Since anomalies were already detected, we hypothesized that K-Means

could group them into distinct categories based on feature similarity.

Method: K-Means aims to partition the anomalies into $k = 3$ clusters, each representing one type of anomaly. The algorithm minimizes intra-cluster distances by iteratively updating cluster centroids.

Results: The output is illustrated in Figure 4.5, which presents a comparison between the real and predicted distribution of anomalies for each category.

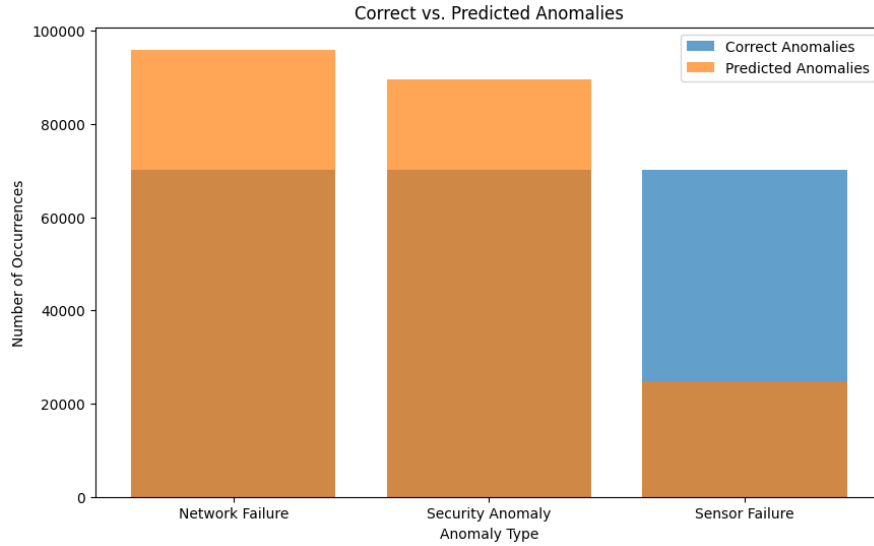


Figure 4.5: Real vs. Predicted Anomalies using K-Means

Observations: K-Means performed well in separating Network Failures and Security Anomalies, with a high degree of overlap between true and predicted labels. However, it significantly misclassified Sensor Failures, likely due to:

- The linear nature of K-Means, which fails to model nonlinear boundaries.
- Overlap in feature space between sensor-related anomalies and others.

Despite being unsupervised, K-Means provided a reasonable starting point, though insufficient for accurate anomaly classification.

4.3.3.2 Random Forest

Motivation: Random Forest, an ensemble method based on decision trees, is known for its high accuracy and resistance to overfitting in many scenarios. It was chosen as our first supervised model due to its interpretability and performance on tabular data.

Method: A Random Forest classifier was trained on a labeled dataset of anomalies. It works by constructing multiple decision trees on random subsets of the data and aggregating their outputs via majority voting.

Results: According to the results depicted in Figure 4.6, we can see that, while training accuracy was extremely high, test accuracy declined considerably. This gap indicated strong overfitting. The model memorized the training examples but failed to generalize to new, unseen data. This could be attributed to:

- Imbalanced distribution of classes (e.g., fewer sensor failures).
- Noise and outliers in the dataset.
- High model complexity without regularization.

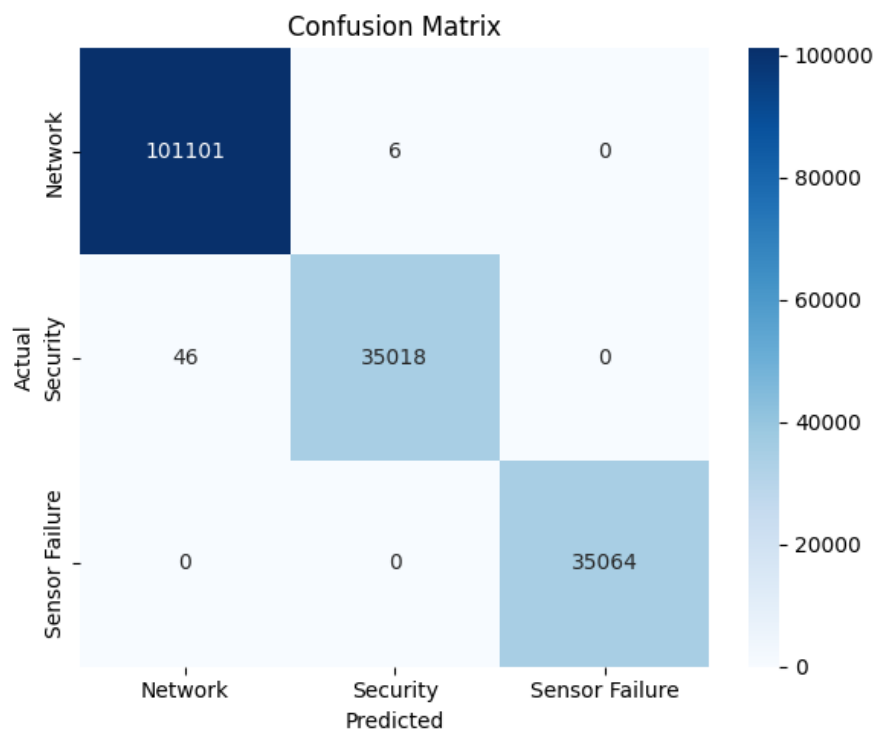


Figure 4.6: Predicted Anomalies using Random Fores

Observations: Although Random Forest can be a powerful tool, in this case it was not effective due to overfitting. Additional techniques such as class balancing, feature selection, or limiting tree depth could improve results, but we opted to proceed with a more robust alternative: XGBoost.

4.3.3.3 XGBoost (Extreme Gradient Boosting)

Motivation: XGBoost is a scalable and regularized version of gradient boosting that tends to outperform other tree-based models on structured datasets. It incorporates both L1 and L2 regularization and supports missing data natively.

Method: The model was trained on the same labeled anomaly dataset. During training, it builds additive trees sequentially, where each new tree corrects the errors of the previous one. Objective functions and evaluation metrics were customized for multi-class classification.

Results: The model is evaluated using the following metrics:

- **Precision:**

$$precision = \frac{TP}{TP + FP}$$

Measures the proportion of correctly identified anomalies out of all detected anomalies.

- **Recall:**

$$recall = \frac{TP}{TP + FN}$$

Represents the proportion of actual anomalies correctly identified.

- **F1-score:**

$$F1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

Harmonic mean of precision and recall.

- **True positives:** the number of data points classified as anomalous that were truly anomalous.
- **False positives:** the number of data points classified as anomalous that were truly non-anomalous.
- **True negatives:** the number of data points classified as non-anomalous that were truly non-anomalous.
- **False negatives:** the number of data points classified as non-anomalous that were truly anomalous.[18]

We implemented these metrics as shown in figure 4.7.

```

# Step 4: Calculate metrics
precision = precision_score(y_binary, y_pred)
recall = recall_score(y_binary, y_pred)
f1 = f1_score(y_binary, y_pred)
accuracy = accuracy_score(y_binary, y_pred)
cm = confusion_matrix(y_binary, y_pred)

# Step 5: Print results
print(f"Threshold: {threshold}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1-score: {f1:.2f}")
print(f"Accuracy: {accuracy:.2f}")
print("Confusion Matrix:")
print(cm)

```

Figure 4.7: calculate metrics

And the evaluation on test data yielded the following results summarized in table 4.1:

Table 4.1: Classification Performance Metrics

Metric	Value
Precision	0.97
Recall	0.67
F1-Score	0.79
Accuracy	0.89

The confusion matrix obtained is as follows:

$$\begin{bmatrix} \text{True Negatives} & \text{False Positives} \\ \text{False Negatives} & \text{True Positives} \end{bmatrix} = \begin{bmatrix} 1,457,652 & 15,037 \\ 210,679 & 420,473 \end{bmatrix}$$

Observations: The high precision indicates that most predicted anomalies were actually correct. However, the recall value of 0.67 reveals that the model missed a portion of actual anomalies. The overall F1-score and accuracy suggest a good balance between precision and recall.

We can conclude that XGBoost provided the best trade-off between accuracy and generalization. It was effective at capturing subtle patterns in the anomaly

data and is well-suited for deployment in real-time IoT anomaly classification pipelines.

4.3.3.4 Two-Stage Classification: XGBoost + SVM

Motivation: After evaluating traditional classifiers, we implemented a two-stage classification approach to further improve accuracy and robustness. The goal was to separate anomalies more effectively by combining the strengths of XGBoost and SVM (Support Vector Machine).

Method:

- **First Stage: XGBoost**

XGBoost is used as a binary classifier to distinguish between **Network anomalies** and **Other anomalies** (Sensor + Security anomalies). It was trained on features derived from the reconstruction errors (mean, std, max, min) of the autoencoder.

- **Second Stage: SVM (Support Vector Machine)**

For instances classified as **Other anomalies**, a second classifier (SVM) is applied to distinguish between **Sensor issues** and **Security anomalies**. The SVM uses the original sensor values and timestamp features for better separation.

The whole picture of the two-stage classification pipeline is described by the figure 4.8 **Architecture Overview:**

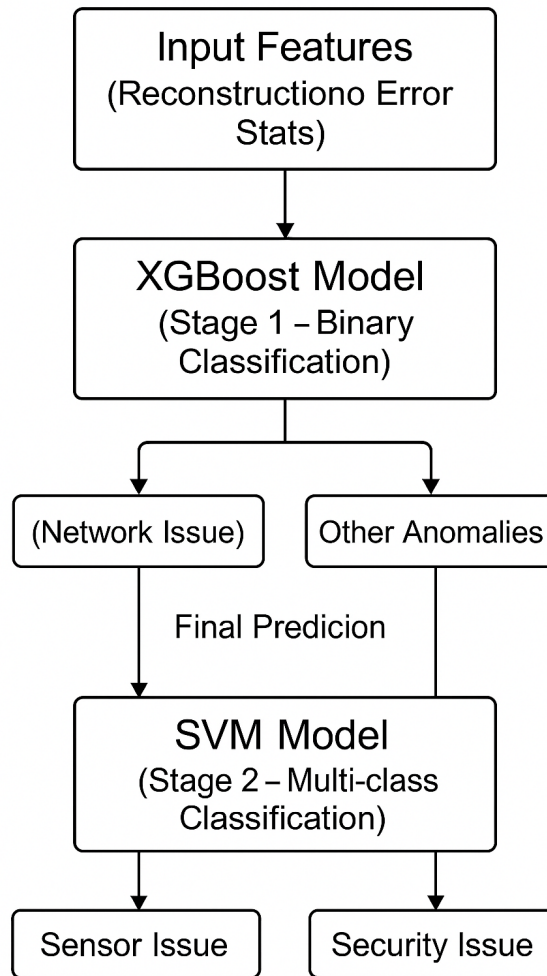


Figure 4.8: Two-Stage Anomaly Classification Architecture (XGBoost + SVM)

Results:

- **XGBoost (Stage 1)** achieved **95% accuracy** in separating network anomalies.
- **SVM (Stage 2)** achieved **90% accuracy** in distinguishing sensor faults from security anomalies.

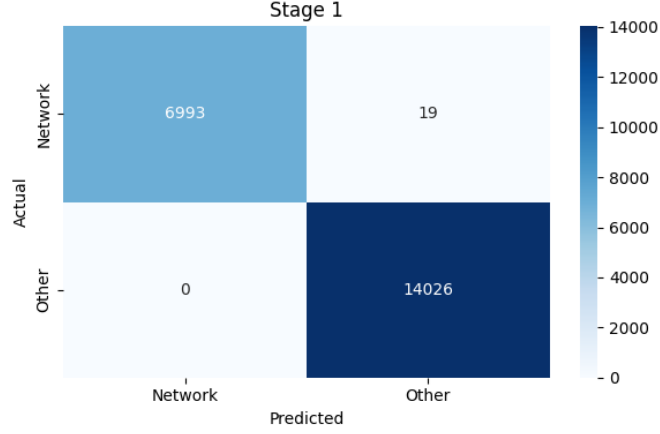


Figure 4.9: XGBoost Confusion Matrix (Stage 1)

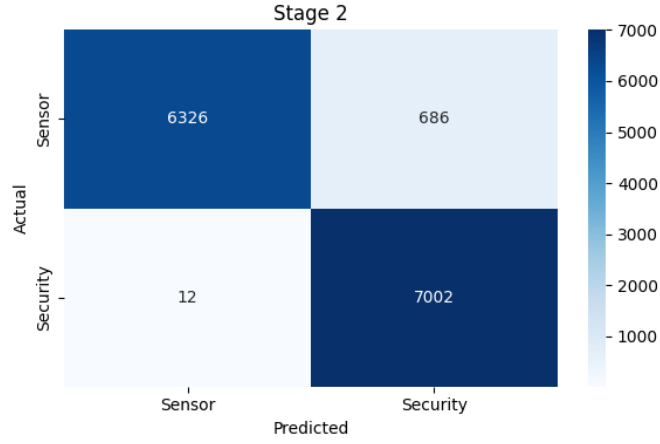


Figure 4.10: SVM Confusion Matrix (Stage 2)

Observations: The two-stage classification pipeline significantly improved the precision and recall compared to using a single model. XGBoost efficiently handled the coarse separation, while SVM captured more nuanced differences between sensor and security anomalies.

4.3.4 Summary

The proposed anomaly detection pipeline successfully distinguishes abnormal behavior in sensor data using autoencoder-based reconstruction error. By introducing labeled anomalies, we enabled both detection and classification. The two-stage

classification (XGBoost + SVM) provided superior performance, achieving high accuracy and robustness compared to individual classifiers.

4.4 The Software Implementation

4.4.1 Technologies Used

To implement our intelligent anomaly detection system, we adopted a combination of modern and efficient technologies across both the frontend and backend. For the backend, we used the ***Django framework***, a high-level Python web framework known for its completeness, security, and scalability. Django allowed us to develop a robust and maintainable application with a clear separation of concerns and strong security practices[19].

On the frontend, standard web technologies such as ***HTML, CSS, and JavaScript*** were used to design interactive and responsive user interfaces. To support machine learning capabilities, we integrated powerful Python libraries including Scikit-learn, TensorFlow, and XGBoost. These tools allowed us to load models capable of detecting and classifying anomalies with high accuracy.

For data storage, we used ***SQLite*** during development for its simplicity, with the option to switch to PostgreSQL in production for enhanced performance and scalability. To visualize the results and insights from the models, we used libraries like ***Plotly, Chart.js, and Matplotlib***, enabling clear and dynamic graphical representations of the system's outputs.

4.4.2 Implementation of the Dashboard

This section describes the user interface developed to interact with the anomaly detection system for IoT sensor networks. The interface was designed and implemented based on the analysis and architecture described in the previous chapters.

4.4.2.1 Authentication Interface

The authentication page 4.11 is the first screen presented to the user. It allows login using an email and password. If the credentials are incorrect, an error message is displayed as depicted in figure 4.12.

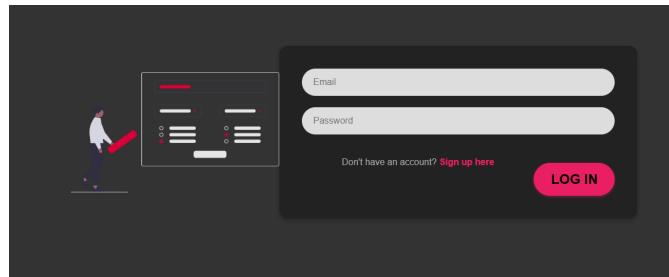


Figure 4.11: Login Interface

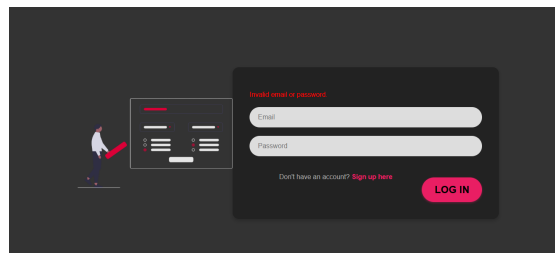


Figure 4.12: Login Error Message

4.4.2.2 Main Dashboard

After a successful login, the user is redirected to the main dashboard shown in figure 4.13. This interface provides a concise overview of the monitoring system. It displays metrics such as the number of active rooms, total sensors, and system status. Users can navigate to other sections like Rooms or History using the sidebar. The central "View Rooms" button guides the user to detailed room activity as visualized by figure 4.14.

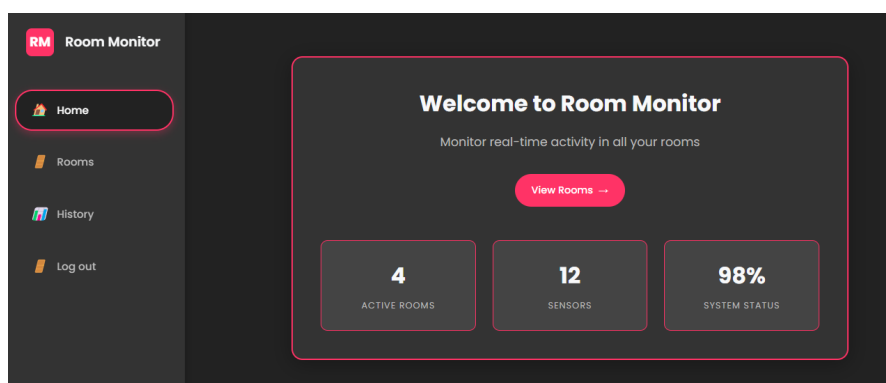


Figure 4.13: Main Dashboard

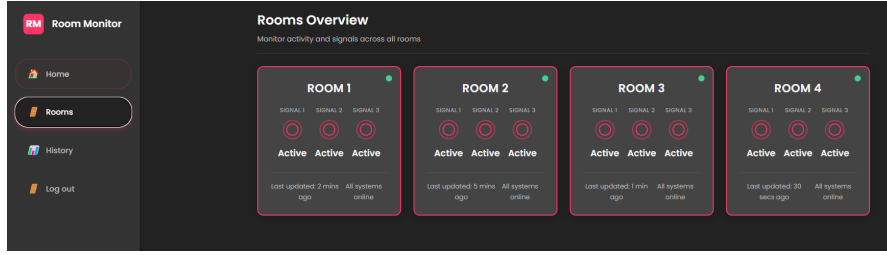


Figure 4.14: Room page

4.4.2.3 Anomaly History Interface

The user can also explore previous anomaly detection sessions. Each record shows the dataset used, the date the anomalies were found, and their types as illustrated in figure 4.15. This helps monitor trends over time and evaluate system stability.

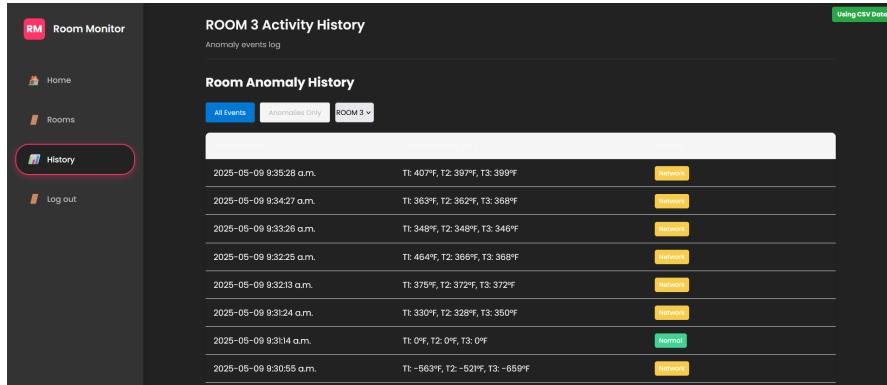


Figure 4.15: Anomaly Detection History

4.5 Conclusion

This chapter presented the experimental evaluation of our anomaly detection system. We combined an autoencoder for anomaly detection with XGBoost for classification, achieving high precision and accuracy. Various anomaly types were simulated and compared using multiple models, with XGBoost showing the best performance. We also developed a web interface for easy data upload and result visualization, making the system accessible and user-friendly for real-world IoT applications.

General Conclusion

This project aimed to design an intelligent system for detecting and classifying anomalies in data collected from IoT sensors. The system combines unsupervised learning techniques (autoencoder) and supervised learning algorithms (XGBoost and SVM). This work addresses a growing need in a context where connected objects generate massive volumes of real-time data, making manual analysis inefficient and impractical at scale.

We followed a structured step-by-step approach. First, the raw data were preprocessed to make it suitable for machine learning models. This included rigorous normalization. Then, an autoencoder was trained on normal data to learn how to reconstruct typical behavior. Any significant reconstruction error was used as an indicator of potential anomalies.

The originality of our approach lies in combining this anomaly detection with a classification step. Although the auto-encoder can signal that an anomaly has occurred, it does not provide any information about the nature of the anomaly. To address this, we implemented a two-stage classification pipeline. In the first stage, an XGBoost model is used to detect and classify network anomalies. In the second stage, any network anomalies are passed to an SVM classifier to determine whether they relate to sensor faults or potential security breaches. This layered design improves the detection accuracy and provides clearer insight into the types of anomalies.

We also paid close attention to the user-facing part of the project. The entire system was integrated into a user-friendly interface developed with Django. This interface allows users to visualize detected anomalies, trigger automatic classification, and see the history of the anomaly. The system is therefore accessible even to nontechnical users, making it suitable for industrial or academic deployment.

Despite the promising results, the project was not without its challenges. One major difficulty was the size of the data set. Due to the high volume of sensor data, training the models, especially the autoencoder, was computationally intensive and time consuming. A second challenge was the initial learning curve associated with the Django web framework. Since this was our first experience with Django, we had to dedicate time to understanding its structure, views, and data flow before

achieving seamless integration with our machine learning pipeline.

The results were encouraging: The autoencoder effectively reconstructed normal data, while the XGBoost and SVM classifiers accurately identified and categorized anomalies. However, improvements are still possible. Expanding the data set with more diverse and real-world anomalies would enhance the generalization of the model. Moreover, deploying the system in a real-time infrastructure with automatic alerts would be a major advancement toward a fully operational solution. As a future perspective, this work could also be extended to not only detect and classify anomalies but also to predict them before they occur. By incorporating temporal modeling techniques such as recurrent neural networks (RNNs), long short-term memory (LSTM) networks, or attention-based architectures, the system could learn temporal dependencies and trends within the sensor data. This would enable proactive maintenance and early warning capabilities, further enhancing the reliability and resilience of IoT deployments in critical environments.

In conclusion, this project has demonstrated the relevance and effectiveness of machine learning in the field of IoT anomaly detection. The hybrid architecture we developed, combining deep learning and traditional machine learning, offers a powerful, scalable, and user-friendly approach that lays the groundwork for future intelligent monitoring systems.

Bibliography

- [1] Fabio Duarte. Number of iot devices (2024). <https://explodingtopics.com/blog/number-of-iot-devices#how-many>.
- [2] Difference between two tier and three tier database architecture.
- [3] Mrs. Jayakeerthi Mrs. Divya Jose J, Dr. D Vimal Kumar. Iot in the modern world. <https://iipseries.org/assets/docupload/rs120244D5417AF50BEBB5.pdf>.
- [4] Satyajit Sinha. State of iot 2024: Number of connected iot devices growing 1318.8 billion globally, 2024. Last accessed 20 October 2024.
- [5] et al. Gubbi, J. Internet of things (iot) : A vision, architectural elements, and future directions. <https://www.scribd.com/document/410532137/1-s2-0-S0045790617323868-main>, 2013.
- [6] et al. Sicari, S. Security, privacy and trust in internet of things: The road ahead. https://www.researchgate.net/publication/270107935_Security_privacy_and_trust_in_Internet_of_Things_The_road_ahead, 2015.
- [7] Banerjee A. Kumar V. Chandola, V. Anomaly detection: A survey. acm computing surveys, 41(3), 1–58. https://www.researchgate.net/publication/220565847_Anomaly_Detection_A_Survey, 2009.
- [8] Arindam Banerjee Varun Chandola and Vipin Kumar. Anomaly detection: A survey. https://www.researchgate.net/publication/220565847_Anomaly_Detection_A_Survey, 2009.
- [9] Park J. M. Patcha, A. n overview of anomaly detection techniques: Existing solutions and latest technological trends. computer networks., 2007.
- [10] Yairi T. Sakurada, M. Anomaly detection using autoencoders with nonlinear dimensionality reduction. <https://dl.acm.org/doi/10.1145/2689746.2689747>, 2014.

- [11] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal. Lstm-based encoder-decoder for multi-sensor anomaly detection. *arXiv preprint arXiv:1607.00148*, 2016.
- [12] M. Munir, S. A. Siddiqui, A. Dengel, and S. Ahmed. Deep autoencoding models for cyber-attack detection in industrial control systems. *arXiv preprint arXiv:1906.01524*, 2019.
- [13] C. Zhou and R. C. Paffenroth. Adversarial autoencoders for anomaly detection in industrial time series. *arXiv preprint arXiv:1709.04107*, 2020.
- [14] H. Xu, Y. Chen, B. Zhao, and H. Li. Unsupervised anomaly detection in time series using variational autoencoders. *arXiv preprint arXiv:1802.03903*, 2018.
- [15] R. Chalapathy and S. Chawla. Anomaly detection using one-class neural networks. *arXiv preprint arXiv:1802.06360*, 2018.
- [16] M. Ahmed, A. Mahmood, and J. Hu. Anomaly detection in industrial iot: A survey. *Journal of Network and Computer Applications*, 2020.
- [17] Zain Nasrullah Yue Zhao and Zheng Li. Pyod: A python toolbox for scalable outlier detection. <https://www.jmlr.org/papers/volume20/19-011/19-011.pdf>, 2019.
- [18] Vesperini F. Eyben F. Squartini S. Schuller B. Marchi, E. A novel approach for automatic acoustic novelty detection using a denoising autoencoder with bidirectional lstm neural networks. *icassp.*, 2015.
- [19] Django Software Foundation. Django documentation, n.d. <https://docs.djangoproject.com/>.